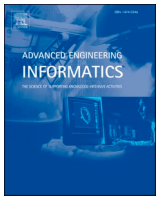
Advanced Engineering Informatics 64 (2025) 103100

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei



Semantic elaboration of Low-LOD BIMs: Inferring functional requirements using graph neural networks

Suhyung Jang [a], Ghang Lee [a,*], Minkyeong Park [a], Jaekun Lee [a], Seungah Suh [a], Bonsang Koo [b]

[a] Architecture and Architectural Engineering, Yonsei University, Seoul, Republic of Korea
[b] Construction Engineering and Management, Seoul National University of Science and Technology, Seoul, Republic of Korea

ARTICLE INFO

Keywords:
Semantic elaboration
Building information model (BIM)
Threshold-enhanced triangle intersection (TETI)
Graph neural network (GNN)
Large language model (LLM) embedding

ABSTRACT

This study proposes a method to automatically subcategorize early object types in low levels of development (LODs) into detailed types (i.e., subtypes) with distinct functional requirements, such as insulation, waterproofing, and load-bearing. While rough cost estimation is possible in the early design phase without detailed object classifications, its accuracy is often limited. Subcategorizing generic objects like walls and columns into more detailed types enhances the precision of early-stage engineering analyses, including cost estimation, load assessments, and material takeoffs. Existing automated object subclassification methods rely on information extracted from highly detailed models, which are unavailable in early-stage building information models (BIMs) due to a lack of geometric and attributive distinctions. This study addresses these limitations by leveraging functional requirements inferred from object connections and placement in early BIMs, achieved using a graph neural network (GNN). To convert BIMs into graphs, a novel threshold-enhanced triangle intersection (TETI) algorithm is introduced, overcoming inaccuracies and exception-handling issues in existing methods. The study explores two GNN-based approaches: node property prediction and node prediction. The former distinguished generic object types into 14 detailed categories, but cost estimation required greater specificity. The latter successfully classified objects into 42 subtypes, with the best results achieved using semantically rich embeddings from a large language model (LLM) and GraphSAGE with three SAGE convolution layers, three hops, and 1,024 dimensions, yielding a weighted F1-score of 0.8766. This approach significantly reduces input data requirements compared to existing methods, enabling more accurate early identification of functional requirements in low-LOD BIMs and supporting both early engineering analyses and detailing processes.

1. Introduction

Project stakeholders often seek to determine the cost and performance of buildings in the early design stages; however, they lack the detailed functional requirements needed to make accurate estimates during this phase. For example, building objects in the building information models (BIM) during the early design stage are generally represented using generic details without specific attributes and subordinate geometries, such as a "generic 200 mm concrete wall" (Fig. 1A). However, each wall object will require distinct functionalities depending on its role: e.g., bathroom walls require *waterproofing,* and exterior walls require *insulation* and *loadbearing* (Fig. 1B). According to the American Association of Cost Engineering (AACE) [1], this lack of detail can lead to cost estimation errors ranging from −30 % to + 50 %. To improve estimate accuracy, several leading contractors have recently started manually subcategorizing BIM objects based on predefined subtypes, specifying the functional requirements and features that influence costs or building performance, such as types of finishes,

Abbreviations: AABB, Axis-Aligned Bounding Box; AACE, American Association of Cost Engineering; BERT, Bidirectional Encoder Representations from Transformers; BIM, Building Information Modeling; CNN, Convolutional Neural Network; GCN, Graph Convolutional Network; GNN, Graph Neural Network; GPT, Generative Pre-trained Transformer; IFC, Industry Foundation Classes; LLaMA, Large Language model Meta AI; LLM, Large Language Model; LOD, Level of Development; MLP, Multi-Layer Perceptron; SAT, Separating Axis Theorem; SVM, Support Vector Machine; TETI, Threshold-Enhanced Triangle Intersection; XGBoost, eXtreme Gradient Boosting.

* Corresponding author.
E-mail address: glee@yonsei.ac.kr (G. Lee).

https://doi.org/10.1016/j.aei.2024.103100
Received 25 July 2024; Received in revised form 22 November 2024; Accepted 30 December 2024

functional locations, and other relevant factors [2,3] (Fig. 1C). While this approach enhances early informed decision-making and improves estimate accuracy, the manual subcategorization of BIM objects is labor-intensive and error-prone [4].

Here, we define the task of inferring detailed information about known objects as *semantic elaboration* [5,6], in contrast to *semantic annotation* [7] which aims to predict unknown BIM objects in common semantic enrichment studies. Several previous studies have addressed semantic elaboration by utilizing rich geometric and attributive information available within high level of development (LOD) BIM objects [8–13]. However, such information is lacking in the case of early design stage BIMs with low LOD. In a preliminary study, we developed a geometric rule-based method to automatically categorize generic wall and slab objects into eight different subtypes-four walls and four slabs subtypes-by different functional requirements [8]. Although the preliminary study achieved an average accuracy of 94.84 %, it also revealed the inherent limitations of a rule-based approach: developing rules for classification is time-consuming, exceptions always exist despite the extended development time, and managing a greater variety of subtypes is required in practice.

In this study, we focus on how human engineers discern BIM objects with low LOD and determine their required functionalities. Human experts understand the roles of building objects and identify their distinct functional requirements based on connectivity and placement. For example, subtypes such as 'Perimeter wall' and 'Side wall' are easily recognized as those forming the external boundaries of a building. These subtypes can be identified in BIMs by their placement and relationship with other BIM objects, even with low LOD. Emulating this expert process, we propose a method to automatically subcategorize low-LOD objects into subtypes by their functional requirements based on their relationships during the early design stage, using a graph neural network (GNN).

Although Wang et al. [15] used a GNN for semantic enrichment, their approach was not solely based on relationships, but also based on attributes and geometries. Moreover, it was aimed at semantic annotation (identifying unnamed objects) rather than semantic elaboration. Additionally, previous studies [15–18] converted a BIM into a graph using *IfcRelationship* after transforming a BIM into an Industry Foundation Classes (IFC) model or by applying a bounding-box collision algorithm. However, these approaches often suffer from missing connections between BIM objects. Moreover, while various subtypes utilized in practice require discerning semantic similarities and differences, conventional encoding methods such as label and one-hot encodings have limitations in capturing such rich semantics.

To overcome these challenges in parsing exact relationships between building components, this paper introduces the threshold-enhanced triangle intersection (TETI) algorithm for accurate yet efficient identification of BIM object connections. In addition, this study proposes a training method utilizing large language model (LLM) embeddings to enable AI models to understand and discern the differences in semantic nuances. The proposed method consists of three main steps: First, generic BIM objects and their relationships are automatically extracted from a BIM using the TETI algorithm. Second, the extracted information is converted into a graph. Finally, the functional requirements of objects are inferred using a GNN, specifically GraphSAGE, trained with semantic nuances augmented by LLM embeddings as data encoding (i.e., LLM encoding).

The proposed method was validated using BIMs from five high-rise residential buildings. These models consisted of six generic object types (i.e., slabs, walls, beams, columns, stairs, and foundations) corresponding to 42 subtypes (e.g., core wall, side wall, roof parapet, and others) associated with three functional requirements. Four experiments were designed to address two different semantic elaboration approaches of functional property prediction and subtype classification. In the first approach, we evaluated the method's effectiveness in predicting three functional properties through the first experiment. In the second approach, we assessed performance in subcategorizing the 42 subtypes through three experiments: 1) determining the optimal configuration of model hyperparameters by examining performance differences based on varying hops (i.e., the number of relationship steps in the graph) and dimensions of each SAGE convolution layer for GraphSAGE, 2) comparing whether incorporating semantic nuances of generic types and subtypes of BIM objects improves performance of semantic elaboration by comparing encoding methods that do not account for semantic

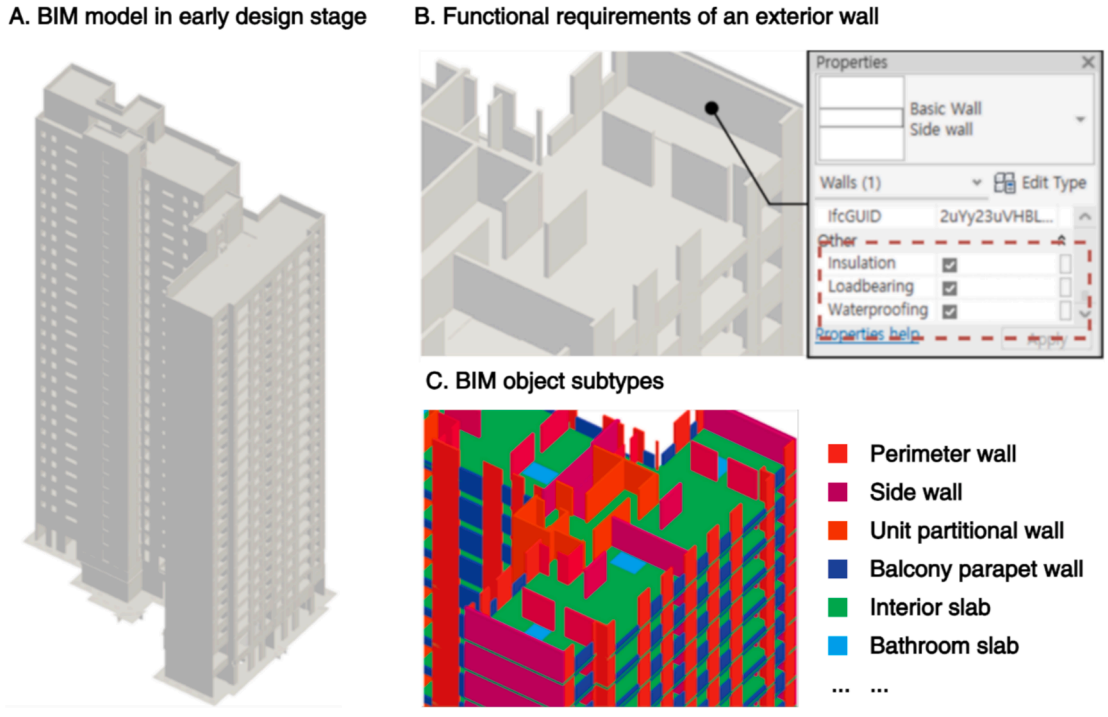


Fig. 1. An example of a BIM in the early design stage and illustration of how functional requirements and subtypes are utilized.

nuances (i.e., label and one-hot encodings) with those that do (i.e., LLM encoding); and 3) comparing the performance of the proposed relation-based GraphSAGE model to other AI models previously used for BIM object subtype classification, including a feature-based machine learning model—support vector machine (SVM) [9,19], an ensemble-learning-based model—eXtreme gradient boosting (XGBoost) [20], and a neural-network-based model—multi-layer perceptron (MLP) [21].

This paper is organized as follows. The next section reviews previous studies and discusses the research gaps. The third section introduces the proposed method in detail. The fourth section explains how the experiments were designed to address the research questions. The fifth section reports the results of the experiments, and the sixth section discusses the results focusing on the types of errors. Finally, the seventh section concludes the paper by discussing the contributions, limitations, and future research directions.

2. Background

2.1. Semantic annotation versus semantic elaboration

Semantic enrichment enhances digital models or data with meaningful information to support informed decision-making, utilizing techniques such as rule-based algorithms [22] and machine learning [23]. Within semantic enrichment, two primary tasks are identified: *semantic annotation* and *semantic elaboration*. Semantic annotation refers to the process of adding semantic information to unlabeled objects, effectively transforming unknown objects into known ones [7]. Extensive studies have been conducted for semantic annotation [9–11,15,18,21,24–27]. In this study, we formulate another semantic enrichment task of inferring detailed information about BIM objects their generic types are known to achieve a higher level of specificity required for informed decision-making. The task is named semantic elaboration borrowing the concepts and terms used in natural language processing. This study focuses on proposing a specific method for semantic elaboration, aiming to automatically infer the functional requirements of BIM objects within early design-stage BIMs, where informed decision-making has a significant impact on the project despite the limited availability of such information. In recent years, extensive research has been conducted on semantic enrichment, encompassing both semantic annotation and semantic elaboration. Table 1 compares the examples of semantic annotation and semantic elaboration, illustrating their respective use cases and objectives.

Table 1
Comparison of semantic annotation and semantic elaboration.

Semantic enrichment types	Use cases	Objectives	References
Semantic Annotation "Unknown to known"	Type identification	To identify object types from properties	[9,11,15,18,21,24,26,27]
		To identify object types from rendered images	[10,12,28]
		To identify object types from point cloud clusters	[25,28]
Semantic Elaboration "Known to informed"	Subtype classification	To classify subtypes of building components	[8–13]
		To classify subtypes of spatial objects	[14,15,23]
	Property prediction	To improve the semantics required for operating and managing built heritages	[29,30]

Semantic annotation is the task or process of adding semantic information to an unlabeled building object, represented by *type identification* that predicts object types based on diverse data sources including geometric properties [9,11,15,21,24,26,27], rendered images [10,12,28], or point cloud clusters [31–34]. *Semantic elaboration* involves appending detailed information (i.e., *property prediction*) or specifying the subcategorization of objects for those whose generic class is known (i.e., *subtype classification*).

Property prediction involves adding detailed properties or metadata to enrich the information required to understand building objects' features in specific tasks. Examples include property prediction for heritage management [29] or early life cycle assessment [30]. Simeone et al. [29] focused on the semantic representation and management of non-geometric information related to heritage buildings, developing an ontology-based system that uses reasoning to predict object properties required for managing these buildings, such as their historical date, state of decay, material, and dimensions. Forth et al. [30] proposed a method to match the material information of the objects in the early design stage BIMs to low-embodied-carbon materials in a life-cycle assessment database using natural language processing. While these methods are effective in enhancing semantic information in the BIM objects, they require the construction of an external database or ontology for semantic elaboration.

Subtype classification involves subcategorizing building objects whose generic types are known, into subtypes that share similar or identical functional properties and roles. At the spatial level, identifying types of rooms corresponds to subtype classification [14,15]. At the building object levels, subtype classification can include diverse cases as presented in Table 2.

Wu and Zhang [11] developed a rule-based algorithm to classify IFC objects into predefined generic types and subtypes based on geometric

Table 2
The number of subtypes addressed in the previous subtype classification studies.

Reference	Generic type	Subtype	Counts	Total
Wu and Zhang, 2019 [11]	Beam	Rectangular Beam, C-Beam, I-Beam, L-Beam, U-Beam, Rectangular Beam with Cuts, Round Beam, Truss, Hollow Round Beam, and Skewed I-Beam	10	10
Kim et al., 2019 [10]	Door	Single, Double, and Revolving	3	9
	Seating	Chair, Office chair, Sofa, and Stool	4	
	Toilet fixture	Toilet and Urinal	2	
Koo et al., 2021 [28]	Culvert	Culvert 1, Culvert 2, and Culvert 3	3	7
	Retaining wall	L-shaped, Semi-gravity, Cantilever, and Gravity	4	
Koo et al., 2021 [12]	Wall	Generic, Window opening, and Wall opening	3	7
	Door	Single, Double, Sliding, and Revolving	4	
Suh, 2023 [8]	Slab	Generic slab, Bathroom slab, Sloped slab, and Flat slab	4	8
	Wall	Wall with aperture, Wall without aperture, Exterior wall, and Interior wall	4	
Liu et al., 2024 [13]	Slab	Slab-A, Slab-B, Slab-C, Slab-D, Slab-E, Slab-F, Slab-G, Slab-H, Slab-I, and Slab-J	10	25
	Beam	Beam-A, Beam-B, Beam-C, and Bream-D	4	
	Column	Column-A and Column-B	2	
	Wall	Wall-A, Wall-B, Wall-C, Wall-D, Wall-E, Wall-F, Wall-G, Wall-H, and Wall-I	9	

features (e.g., the number of lines, curves, and angles), achieving 100 % accuracy for all subtypes. Kim et al. [10] proposed an image-based approach for classifying generic object types and subtypes by employing 2D convolutional neural networks (CNNs) to process rendered images of BIM objects captured from multiple views. This method identified eight generic object types and subclassified door, seating, and toilet fixtures. Koo et al. [28] compared computer vision and point-cloud processing for classifying and subcategorizing BIM objects in road infrastructure. They trained multiple-view CNN [35] for the computer-vision approach, and PointNet [36] for point-cloud processing. Koo et al. [12] extended their study by including a support vector machine to classify wall and door objects by geometric features. Liu et al. [13] introduced a multi-modal deep learning approach for subcategorizing BIM objects to support cost estimation, discerning 25 subtypes across four generic object types by combining visual and textural attributes. They employed a CNN-based model for visual features and an attention-based approach for textual attributes.

While these methods successfully subcategorize BIM object subtypes based on their geometric, visual, and semantic features, they commonly focus on utilizing rich features available at high LOD. Conversely, such features are absent in early design stage BIMs with low LOD. Furthermore, the number of subtypes addressed in the previous studies is typically limited to ten, while the subtypes used in practice are more diverse, with 42 subtypes employed in this study. Although discerning such diverse functional roles of building objects is complex, humans can infer functionalities based on object placement within the model. Emulating this human ability, this study aims to propose a method to convert BIMs into graphs to represent their relationships and utilize GNNs to semantically elaborate on functional requirements based on these relationships.

2.2. Graph representation of BIMs

Converting BIMs into graphs to represent their connections is not a noble idea; however, previous methods used to extract the connections between BIM objects present several limitations when applied to complex BIMs. A summary of previous efforts in converting the BIMs into graphs is provided in Table 3.

Collins et al. [37] proposed a method to encode BIM objects into graphs by transforming the geometries of IFC instances into triangular meshes and then converting the mesh vertices into graph nodes and the mesh edges into graph edges. They employed graph convolutional networks (GCN) and dynamic graph CNN to classify the generic industry foundation classes (IFC) classifications of BIM objects. Although these mesh representations accurately capture the boundaries of each building object, the method does not address identifying intersections between multiple building objects.

Table 3
Previous studies represented BIMs as graphs.

Reference	Objective	Node	Edge
Khalili and Chua, 2015 [16]	To propose a graph representation of the BIM that can represent contextual relationships among building components	Rooms and building component	*IfcRelationship*
Skandhakumar et al., 2016 [17]	To propose a graph representation of the BIM for access control applications.	Building, floors, and rooms	*IfcRelationship*
Collins et al. 2021, 2022 [37]	To classify IFC instances from their surface mesh representation.	Vertices of a surface mesh	Edges of a surface mesh
Wang et al. 2022 [14]	To subcategorize rooms of apartment layout.	Room	Manually parsed relationship
Wang et al. 2024 [15]	To propose a graph-based common data environment of BIMs.	Site, building, level, space, and building components with attributes	Bounding-box-based adjacency
Austern et al. 2024 [18]	To annotate generic object types from their connections.	Bounding boxes of building components and dimensions.	Bounding-box-based adjacency

Khalili and Chua [16] and Skandhakumar et al. [17] utilized predefined relationships in IFC, specifically *IfcRelationship*. However, *IfcRelationship* entities are only created when BIM objects are explicitly constrained to one another during the authoring process, such as when walls are modeled continuously or extended to attach to slabs. As shown in Fig. 2, although the target wall geometrically collides with multiple slabs within the BIM, it is only connected to the walls on each side through *IfcRelationship*. In such cases, *IfcRelationship* cannot represent the correct geometric connections needed to capture the connections of BIM objects within BIMs.

Wang et al. [14] classified room types in residential complexes using an improved GraphSAGE, comparing it with GCN, multi-layer perceptron (MLP), and decision tree methods. In their study, the authors manually parsed BIM objects such as rooms, walls, windows, and doors, along with their relationships, from 2D and 3D plans. They used bounding boxes to parse the adjacency of building and spatial objects. Similarly, Austern et al. [18] proposed a method to convert BIMs into graphs based on the bounding boxes of building objects. Using GNN, they classified the generic types of building objects from connected geometries whose classes are not known (i.e., semantic annotation). While these bounding box-based approaches do not rely on constraints generated during the authoring process, relying on identifying collisions between bounding boxes may fail to reflect accurate connectivity between BIM objects, as shown in Fig. 3.

As depicted in Fig. 3A and B, BIM objects that do not align with the global X and Y axes of the BIM authoring tool will produce bounding boxes enlarged due to grid rotation. This causes BIM objects that are parallel to each other to be detected as colliding. Even when a minimum bounding box (i.e., a bounding box rotated according to the objects' relative axis) is used, representing relations incorporating non-box-shaped BIM objects, such as slabs in Fig. 3C and D, still present challenges. In these cases, even if the target and candidate objects are not adjacent in reality, as in Fig. 3C, the bounding box may expand the boundaries, as in Fig. 3D, causing the candidate object to be incorrectly detected as colliding with the target object. An ideal solution for reflecting the connections of BIM objects within BIMs as graphs would involve parsing collisions based on their geometric intersections and using them as graph edges. To address this issue, this study introduces a novel algorithm—the TETI algorithm—that accurately detects collisions based on the intersection of BIM objects' geometries, thereby overcoming the limitations of previous methods. The details of the TETI algorithm are further explained in subsection 3.1.

2.3. Encoding methods of semantic information

Understanding semantic differences between generic types and subtypes is crucial for experts to discern the functional requirements of building objects as exemplified by bathroom walls requiring *waterproofing*, and exterior walls requiring *insulation* and *loadbearing*. While human experts can easily discern the differences between such subtypes and decide on required functionalities, conventional encoding methods such as label and one-hot encodings [38] cannot efficiently deliver these semantic nuances. Fig. 4 depicts how the Euclidean distances between the generic types are distributed when the label and one-hot encodings are used to represent the generic BIM object types.

Label encoding converts categorical variables into sequential numeric labels, resulting in varying distances based on the label sequence. In contrast, one-hot encoding represents categorical variables

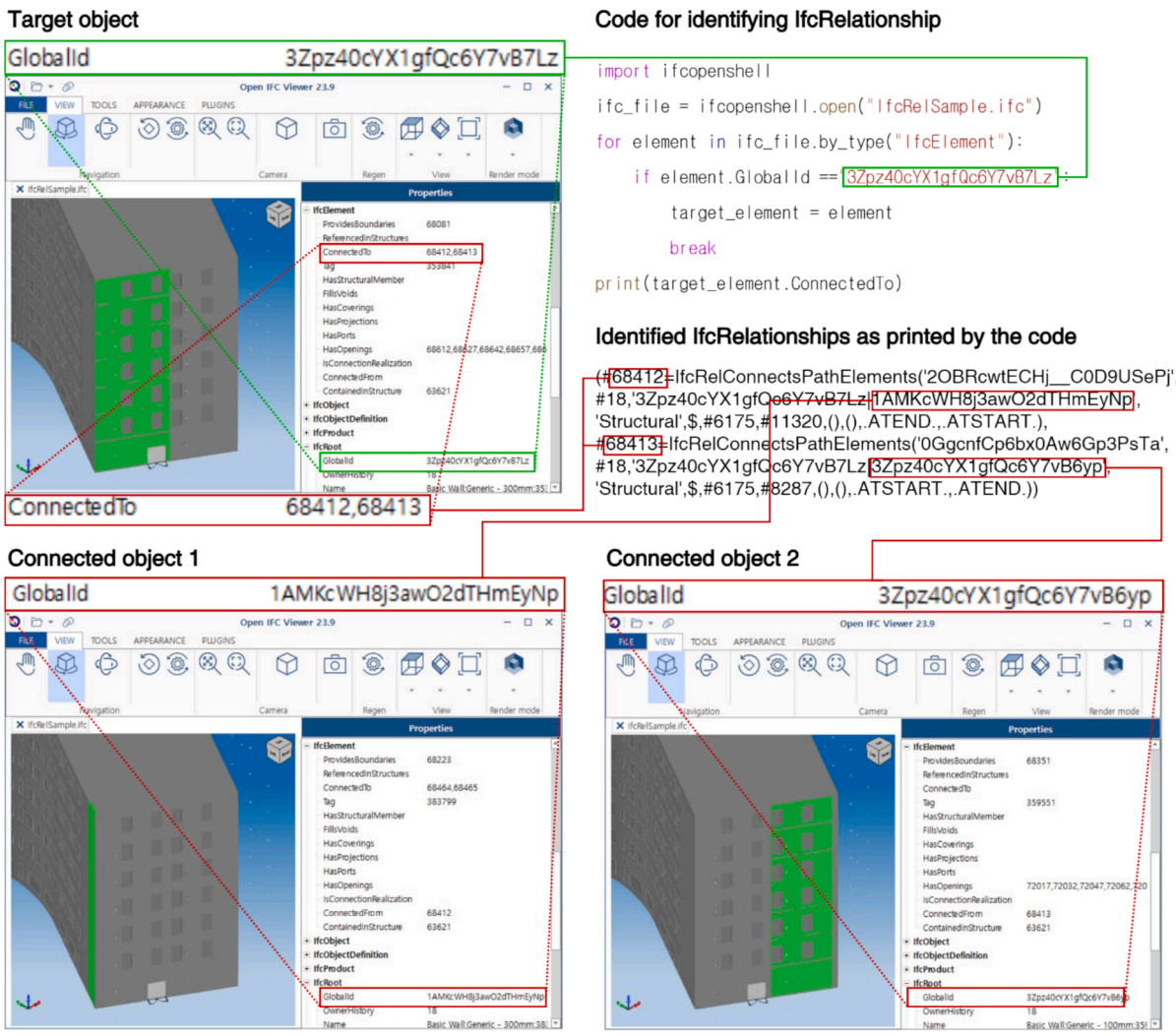


Fig. 2. Adjacency between building objects identified using IfcRelationship.

as binary vectors, with a vector size equal to the number of categories. This approach maintains a consistent distance between different categories [39]. However, this uniformity and lack of relational information may not efficiently reflect the similarities and dissimilarities between categories.

In contrast, novel LLM embeddings leverage advanced LLMs, such as bidirectional encoder representations from transformers (BERT) [40], generative pre-trained transformers (GPT) [41], and large language model Meta AI (LLaMA) [42], to generate highly contextualized vector representations of texts that reflect semantic nuances. Fig. 5 illustrates how the LLM embedding reflects semantic distances between generic types and subtypes of building objects, using t-SNE visualization [43] on OpenAI's 'text-embedding-3-large', which has a vector size of 3,720. For instance, nuanced features of generic types such as 'Beam' and 'Column' have more similar contextual roles compared to 'Stair', while diverse wall subtypes are clustered along the generic type 'Wall'.

Despite several studies being conducted to integrate LLMs into the BIM process [44–49], utilizing LLM embeddings to enhance the semantic understanding of BIMs remains unexplored. This study hypothesizes that providing these semantic nuances as encodings for training AI models for semantic elaboration can affect the performance of the proposed method. By incorporating context-rich LLM embedding as data encoding, we aim to facilitate more accurate identification of the functional requirements of BIM objects.

2.4. Research gaps and questions

The current research gaps can be summarized as follows. First, the previous studies conducted for subtype classification rely on rich geometric, visual, and semantic features, while such detailed features are absent in early design phase BIMs. Second, while utilizing GNNs to subcategorize room elements based on contextual information has been proposed, subcategorizing contextual roles and corresponding functional requirements of building objects is a more complex task that has not been addressed in the previous studies. Third, to extract relationships between BIM objects, previous studies utilized *IfcRelationship* in an IFC model or bounding-box-based collision tests. However, they often miss or incorrectly identify relationships. Last, while discerning semantic nuances are significant in distinguishing the contextual role and functionalities of BIM objects, conventional encoding methods such as label and one-hot encodings fail to capture the inherent similarities and dissimilarities between various object subtypes, potentially hindering the performance of AI models in semantic elaboration tasks.

To address these gaps, this study aims to propose a method that leverages the accurate relationships between BIM objects and incorporates semantic nuances using LLM embedding as data encoding (i.e., LLM encoding). We employ two approaches to infer the functional requirements. The first is property prediction, which evaluates the ability to predict functional properties (i.e., insulation, waterproofing, and loadbearing) of BIM objects based solely on their relationships

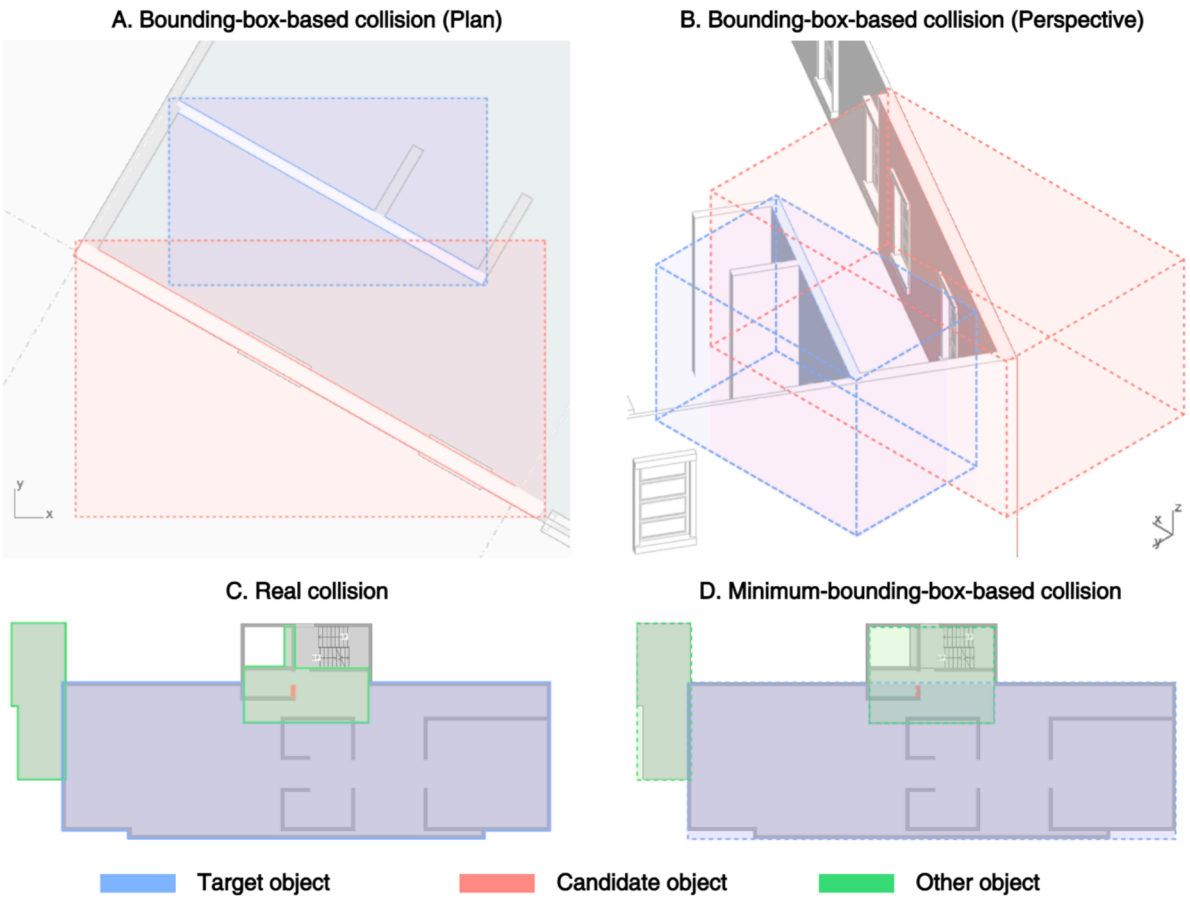


Fig. 3. Fault collision detection when bounding boxes are utilized for identifying relations between building objects.

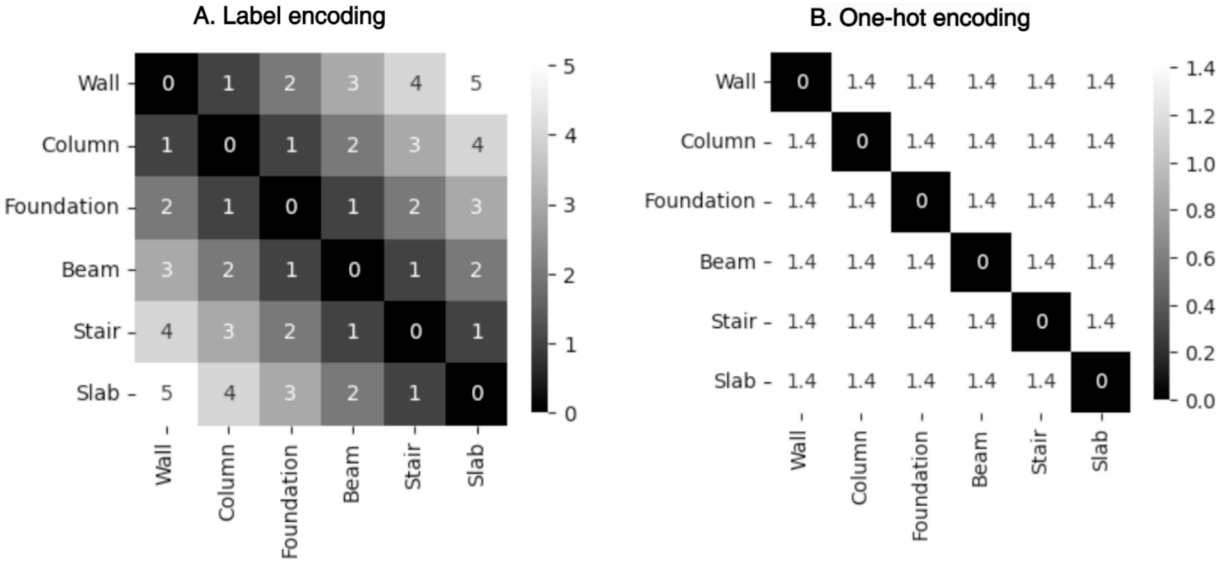


Fig. 4. Euclidean distance between labels represented in label and one-hot encodings.

within the model. The second is subtype classification, which assesses the performance in subcategorizing 42 subtypes of BIM objects. In this approach, we explore optimal configurations of model hyperparameters, incorporate semantic nuances using LLM encoding to verify if it could increase the performance of semantic elaboration, and compare the proposed GNN-based method with other AI models utilized for subtype classification in previous studies. Under this context, the study aims to answer the following research questions:

1) How effective is the proposed method in the semantic elaboration of BIM objects in early design stage BIMs based solely on their relationships compared to traditionally used non-graph-based subtype classification models, such as SVM, XGBoost, and MLP?

Fig. 5. t-SNE visualization of LLM embeddings on generic types and subtypes.

2) What is the optimal configuration of model hyperparameters (e.g., number of hops and dimensions) in the context of subtype classification of BIM objects?
3) Does incorporating semantic nuances of building object categories and subtypes into a GNN using LLM encoding improve the performance of semantic elaboration in both property prediction and subtype classification compared to traditional label and one-hot encodings?

By exploring these questions, this study aims to develop a robust method to enhance semantic elaboration for early design-stage BIMs, enabling more accurate identification of functional requirements and contextual roles (i.e., subtypes) of BIM objects based solely on their relationships. This approach supports informed decision-making during the critical early design stages. The following sections outline the proposed method and its implementation for validation experiments.

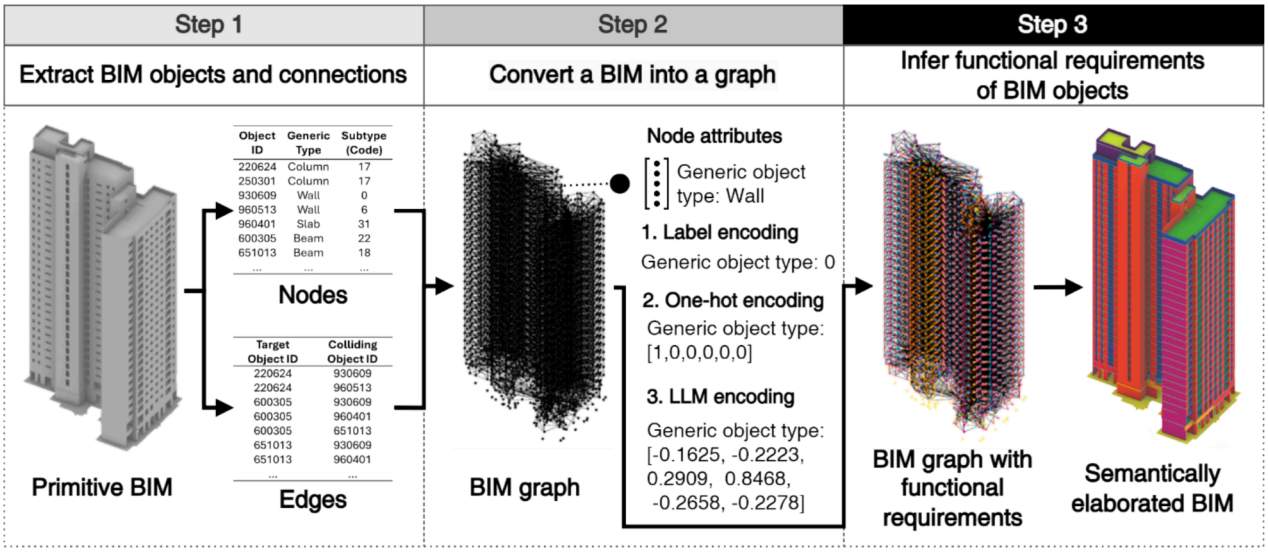


Fig. 6. The proposed method for inferencing functional requirements based on BIM object connections using GNN.

3. Research method

The proposed method consists of three steps, as depicted in Fig. 6. In the first step, building objects and their connections are automatically extracted from a BIM using the TETI algorithm. In the second step, the extracted BIM data is converted into a graph, representing instances of building components as nodes and connections as edges. The third step predicts and applies the functional requirements of the BIM objects using a GNN model.

Before explaining the method in detail, it may be necessary to discuss the scope of the functional requirements targeted for semantic elaboration in this study. This study focuses on inferring the functional requirements of six generic BIM object types, namely, wall, column, beam, stairs, slab, and foundation (Fig. 7). In this study, we assume that the geometric shapes of the generic objects will remain "undetailed (thus, generic)" in the early design phase, necessitating the prediction of their functionalities (or subcategorizing them into target subtypes).

With the help of a BIM team from a top general contractor in South Korea, 42 subtypes with logical combinations of functionalities for high-rise residential buildings were identified. These properties include vertical location (from substructure to ground floor, typical floor, penthouse, and roof), horizontal location (from residential spaces such as bedroom, bathroom, and balcony to service spaces such as entrance, core, ramp, and pit, and auxiliary spaces such as senior, day-care, and community centers), and functional requirements (insulation, waterproofing, and structural requirements) as binary classifications. Theoretically, a combination of these properties can result in thousands of subtypes of the six generic objects. However, in practice, the types of subcategorized objects used in a building are quite limited. Codifying the subtypes so their properties can be collectively managed is most preferred in practice, as shown in Fig. 8.

The first coding scheme deploys a strategy to list all major factors that affect early cost estimation and performance analysis: generic object type, vertical location, horizontal location, and functional requirements (insulation, waterproofing, and loadbearing requirements). Mathematically, this combination could generate over 1,000 codes. However, knowing that, for example, a slab is located under a bathroom, the functional requirements of the slab could be logically derived, i.e., that no insulation is required while waterproofing and loadbearing are required. The second coding scheme was generated by merging the horizontal location and the functional requirements. After learning that high-rise residential buildings need only 42 functional codes to distinguish the topological and functional differences between generic objects, the coding scheme was further simplified to named subtypes and corresponding Arabic numbers (i.e., subtype code). Table 4 lists the functionalities required for the objects categorized as each subtype.

3.1. Data Extraction – TETI algorithm

To construct graph data from the BIM, we developed algorithms to extract building objects as nodes and their adjacencies as edges. For nodes, information such as identifier (ID), generic type, and subtype is extracted from the properties of BIM objects using Algorithm 1

Algorithm 1: Object (Node) Data Extraction Algorithm

Input: A BIM
Output: Nodes **with** their IDs, generic types, **and** subtypes extracted **from** the BIM

```
m ← Open(BIM)
Initialize an empty list of nodes
For i = 0 to number of objects in m-1:
    object ← m.getObject(i)
    If object.genericType is in ['Wall',
    'Slab','Column','Beam','Stairs','Foundation']:
    Initialize a new node
    ID ← object.ID
    genericType ← object.genericType
    subtype ← object.subtype
    node ← (ID, genericType, subtype)
    Add node to the list of nodes
End For
Save and close the list of nodes
```

To construct the edges (relationships) between nodes in the graph, we introduce the TETI algorithm developed in this study. The TETI algorithm enables efficient and exact detection of geometric collisions between building objects within a specified threshold. It extends the separating axis theorem (SAT) for triangle-triangle intersections proposed by Möller [44], which iteratively identifies intersections between subordinate convex shapes (i.e., triangles composed of meshes) to detect collisions between meshes.

The SAT is used in computational geometry to determine whether two convex shapes intersect. According to SAT, two convex shapes do not intersect if there exists a separating axis on which their projections do not overlap. For triangle-triangle intersection detection in three-dimensional space, SAT identifies potential separating axes and checks for overlap in projections. Given two triangles, $A = \triangle(p_1, q_1, r_1)$ and $B = \Delta(p_2, q_2, r_2)$, the candidate separating axes include the normal vectors of each triangle's plane, n_A and n_B:

$$n_A = (q_1 - p_1) \times (r_1 - p_1), n_B = (q_2 - p_2) \times (r_2 - p_2)$$

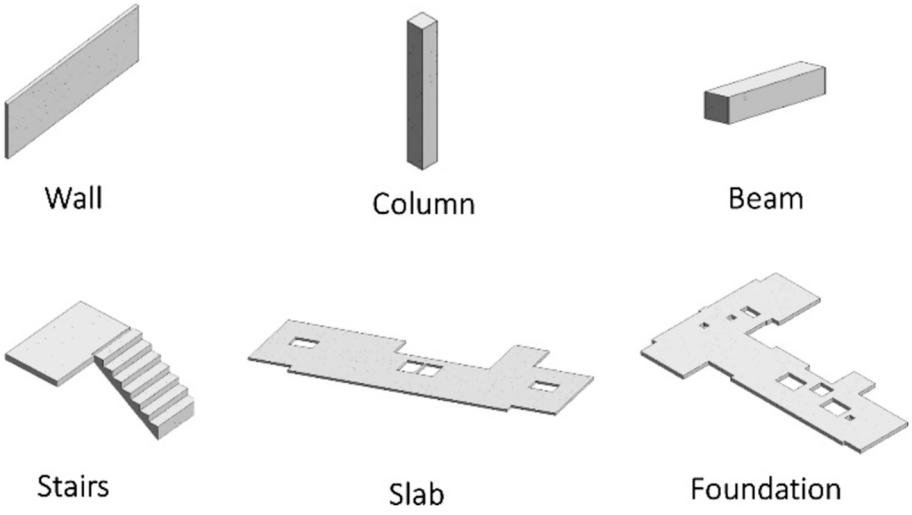


Fig. 7. Examples of generic BIM object types.

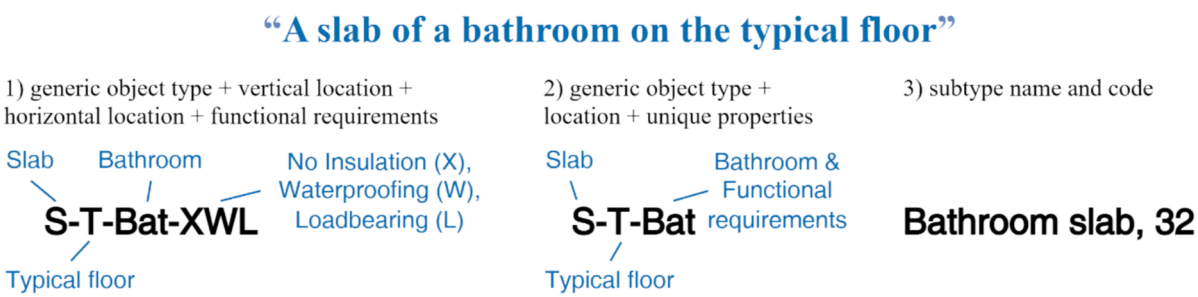


Fig. 8. Three coding schemes considered for function code.

Table 4
Functional requirements code for objects with different functional requirements.

Code	Generic Type	Subtype	Functional Requirement		
			Insulation	Waterproofing	Loadbearing
0	Wall	Core wall	–	–	Required
1	Wall	Horizontal wall	–	–	Required
2	Wall	Vertical wall	–	–	Required
3	Wall	Perimeter wall	Required	Required	Required
4	Wall	Side wall	Required	Required	Required
5	Wall	Entrance retaining wall	–	–	Required
6	Wall	Loadbearing retaining wall	–	–	Required
7	Wall	Miscellaneous wall	–	–	Required
8	Wall	Unit partition wall	–	–	–
9	Wall	Non-loadbearing wall	–	–	–
10	Wall	Auxiliary space wall	–	–	Required
11	Wall	Roof ornamental wall	–	–	–
12	Wall	Roof parapet wall	Required	Required	–
13	Wall	Penthouse parapet wall	Required	Required	–
14	Wall	Ramp parapet wall	–	–	Required
15	Wall	Balcony parapet wall	–	–	–
16	Wall	Miscellaneous parapet wall	–	–	–
17	Column	Transfer column	–	–	Required
18	Beam	Core beam	–	–	Required
19	Beam	Transfer beam	–	–	Required
20	Beam	Miscellaneous beam	–	–	Required
21	Beam	Wall girder	–	–	Required
22	Beam	Interior lintel	–	–	Required
23	Beam	Non-loadbearing lintel	–	–	–
24	Stairs	Entrance stairs	–	–	Required
25	Stairs	Interior stairs	–	–	Required
26	Slab	Core slab	Required	–	Required
27	Slab	Entrance slab	–	–	Required
28	Slab	Entrance ramp slab	–	–	Required
29	Slab	Piloti slab	–	–	Required
30	Slab	Basement utility pit slab	–	Required	Required
31	Slab	Interior slab	–	–	Required
32	Slab	Bathroom slab	–	Required	Required
33	Slab	Miscellaneous slab	–	–	Required
34	Slab	Auxiliary space slab	–	–	Required
35	Slab	Penthouse interior slab	Required	Required	Required
36	Slab	Penthouse core slab	Required	Required	Required
37	Slab	Roof ornamental slab	–	–	–
38	Slab	Canopy slab	–	–	Required
39	Foundation	Entrance strip foundation	–	–	Required
40	Foundation	Mat foundation	–	–	Required
41	Foundation	Haunch	–	–	Required

are the cross products of edges from each triangle, for each edge is $e_{i,A}$ of A and $e_{i,B}$ of B:

$$a_{ij} = e_{i,A} \times e_{j,B}$$

where:

$$e_{1,A} = q_1 - p_1, e_{2,A} = r_1 - p_1, e_{3,A} = r_1 - q_1,$$

$$e_{1,B} = q_2 - p_2, e_{2,B} = r_2 - p_2, e_{3,B} = r_2 - q_2$$

For each axis a (either n_A, n_B, or a_{ij}), project the vertices of A and B onto a, the project of a vertex v onto a is:

$$\text{proj}_a(v) = \frac{v \bullet a}{\|a\|}$$

The projection interval I_A and I_B for triangles A and B on a are:

$$I_A = [min(proj_a(p_1), proj_a(q_1), proj_a(r_1)), max(proj_a(p_1), proj_a(q_1), proj_a(r_1))]$$

$$I_B = [min(proj_a(p_2), proj_a(q_2), proj_a(r_2)), max(proj_a(p_2), proj_a(q_2), proj_a(r_2))]$$

If there exists any axis a where the intervals I_A and I_B do not overlap,

defined as:

$$max(I_A) < min(I_B)) \cup (max(I_B) < min(I_A)$$

then the triangles are non-intersecting.

While the SAT algorithm can identify exact collisions between meshes, it requires checking all possible triangle pairs between the mesh geometries of BIM objects, which is computationally intensive. The TETI algorithm enhances the efficiency of SAT by incorporating broad-phase bounding box collision detection and a threshold to reduce the number of candidate triangle pairs and to detect near-miss interactions commonly found in BIMs.

For broad-phase collision detection, the TETI algorithm identifies intersections between axis-aligned bounding boxes (AABBs). An AABB is a simple representation that encloses an object within minimum and maximum coordinates along the x, y, and z axes. For two objects, A and B, their AABBs overlap if their bounds along the x, y, and z axes overlap. This means that if the range of coordinates for object A overlaps with the range of coordinates for object B on all three axes, the bounding boxes intersect. For object pairs where the bounding boxes collide, the TETI algorithm detects geometric collisions using the SAT enhanced with threshold ε. The threshold is incorporated in checking the magnitude of mesh triangles' normal vectors, handling nearly coplanar triangles, and detecting near-intersections.

If the magnitude of a triangle's normal vector is less than ε, the triangle is considered degenerate (i.e., not accounted for collision checks). For two triangles A and B with normal vectors n_A and n_B, the condition is:

$$\|n_A\| < \varepsilon \; or \; \|n_B\| < \varepsilon$$

If either condition is true, the triangles are treated as degenerate, and no intersection is recorded, improving computational efficiency. Additionally, the TETI algorithm further enhances efficiency by handling nearly coplanar triangles. It does this by checking where the magnitude of each cross-product a_{ij} is smaller than ε:

$$\left\|a_{ij}\right\| < \varepsilon$$

In cases where the triangles are coplanar, TETI bypasses further collision checks and performs a two-dimensional overlap check.

To detect near-intersections, the TETI algorithm expands the projection intervals by ε along each candidate axis. For a given axis a, the adjusted projection intervals for triangles A and B are:

$$I_A^\varepsilon = [min(I_A) - \varepsilon, max(I_A) + \varepsilon]$$

$$I_B^\varepsilon = [min(I_B) - \varepsilon, max(I_B) + \varepsilon]$$

If the intervals I_A^ε and I_B^ε do not overlap, defined as:

$$(max(I_A) - \varepsilon < min(I_B) + \varepsilon) \cup (max(I_B) - \varepsilon < min(I_A) + \varepsilon)$$

then the triangles are non-intersecting. By expanding the intervals, the algorithm accounts for near-miss interactions where objects are geometrically very close but do not intersect, which is a BIM system it is common if modelers do not constraint BIM objects while authoring or also from rounding errors from how computers handle floating-point arithmetic.

Using the TETI algorithm, we extract accurate geometrical adjacency between building components to parse edges. The edges are created by identifying pairs of BIM objects that are connected, as described in Algorithm 2. For implementation, we utilized the rhinoscriptsyntax library in Python as a component within a visual programming environment (i.e., Rhino.Inside.Revit and Grasshopper) in Autodesk Revit. This setup enables the direct extraction of nodes and edges of graphs from BIMs. The threshold value in our study was set to 10 mm, but it can be adjusted in other applications. The code for the TETI algorithm can be accessed at the following link: https://github.com/RGB000KR/TETI.

Algorithm 2: Connection (Edge) Data Extraction Algorithm – TETI Algorithm

Input: A BIM, threshold ε
Output: Lists target_IDs **and** colliding_IDs containing indices of colliding objects IDs

```
m ← Open(BIM)
Initialize empty lists objects, target_IDs and colliding_IDs
For i = 0 to number of objects in m-1:
    object ← m.getObject(i)
    If object.genericType is in ['Wall','Slab','Column','Beam','Stairs','Foundation']:
    ID ← object.ID
    mesh ← object.mesh
    boundingBox ← object.boundingBox
    object ← (ID, mesh, boundingBox)
    Add object to the list of objects
End For
For i = 0 to number of object-1:
    target_object ← objects.getObject(i)
    target_ID ← target_object.ID
    For j = 0 to number of objects-1:
    candidate_object ← objects.getObject(j)
    candidate_ID ← candidate_object.ID
    If i ≠ j and BoundingBoxesIntersect(target_object.boundingBox,
    candidate_object.boundingBox) = True:
    If MeshCollision(target_object.mesh, candidate_object.mesh, ε) = True:
    Add target_ID to the list of target_IDs
    Add candidate_ID to the list of colliding_IDs
    End For
End For
Save and close the list of target_IDs and colliding_IDs
```

3.2. Conversion into a graph

The graph was constructed using BIM objects as nodes with generic types and subtypes as attributes, and their topological connections as edges as illustrated in Fig. 9. ID, generic type, subtype, and functional properties required for each subtype, as listed in Table 4, are added as node properties. While the application does not require including subtypes and functional properties, they were extracted as ground truth labels for training. Connections detected from the previous step are utilized as edges which connect objects with corresponding IDs. When converting the generic types and subtypes we employed varying encoding types including label, one-hot, and LLM encodings. For LLM encoding, we utilized OpenAI's 'text-embedding-3-small' and 'text-embedding-3-large' models [45]. The original embedding sizes of the 'text-embedding-3-small' and 'text-embedding-3-large' are 1,536 and 3,072 dimensions, respectively, which we employed using Matryoshka representation learning [46], a technique known for preserving significant semantic information while reducing dimensionality to comply with the layer size of the GraphSAGE. The NetworkX library in Python was used to convert the extracted data into graph format for GNN input.

3.3. Semantic elaboration using GraphSAGE

In this study, we utilized GraphSAGE, proposed by Hamilton et al. [47], which incorporates a neighborhood sampling strategy before the aggregation stage, arbitrarily sampling a fixed number of neighboring nodes of a target node. Unlike other GNNs, GraphSAGE supports inductive learning, making it suitable for practical scenarios where the structure of a graph for a new building project is unknown during the training stage. This inductive capability allows the model to generalize to unseen nodes and graphs, which is critical as new projects continually introduce unique and previously unobserved graph structures. The conventional GraphSAGE model processes data through multiple layers, using neighborhood aggregation to update node representations iteratively. The GraphSAGE can be formulated as:

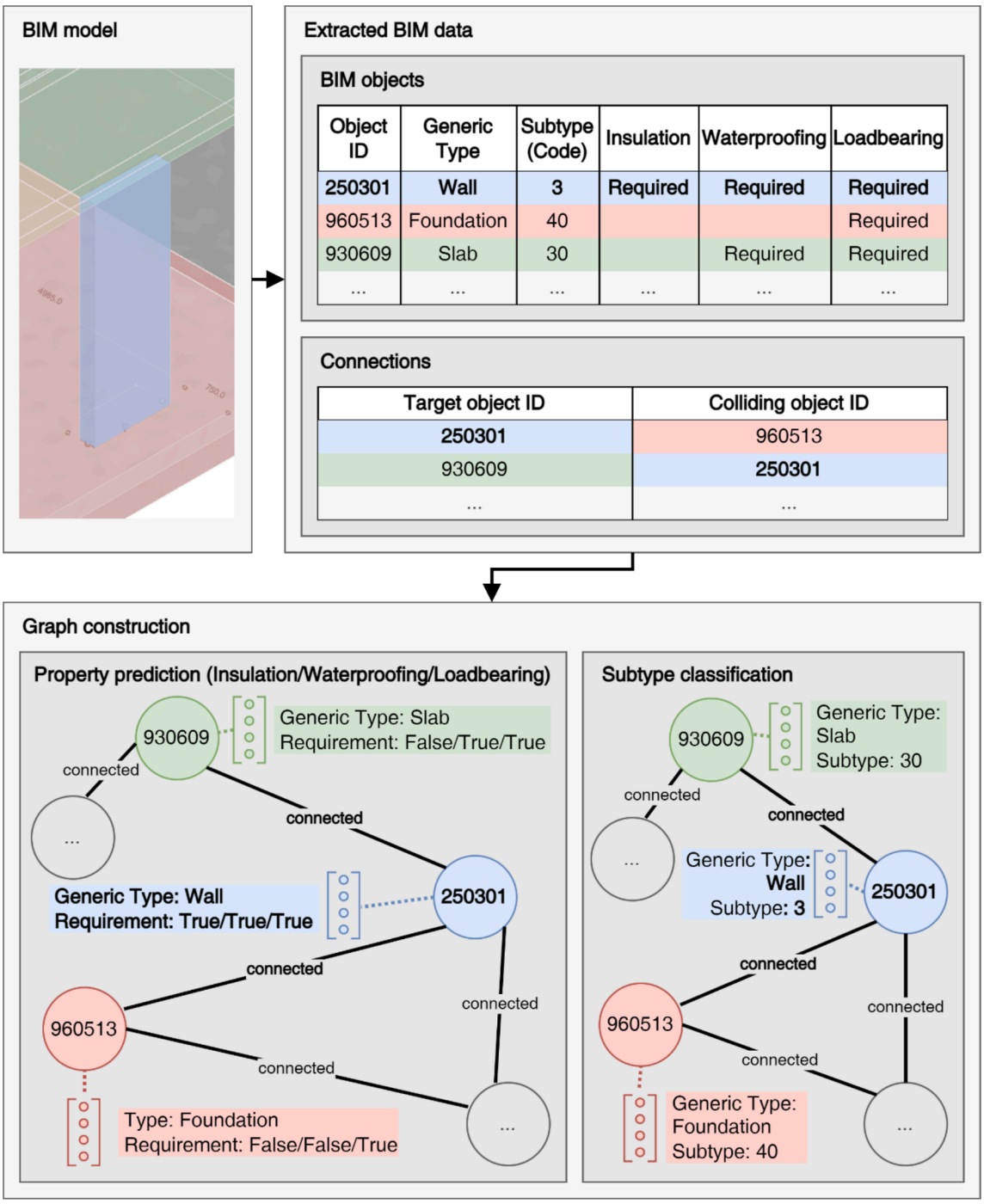


Fig. 9. BIM to graph conversion.

$$h_u^{(k)} = \sigma\left(W \bullet MEAN\left(\left\{h_u^{(k-1)}\right\} \cup \left\{h_v^{(k-1)}, \forall v \in \mathcal{N}(u)\right\}\right)\right)$$

where k is the iteration step; $h_u^{(k)}$ is the hidden embedding vector of each node u; v is the nodes in u's neighborhood $N(u)$; W is the weight matrix; and σ denotes a nonlinear activation function. The iteration step k in the GraphSAGE architecture corresponds to the number of hops of neighboring nodes being considered.

By aggregating the features of neighboring nodes, GraphSAGE not only considers edges (i.e., topological connectivity) but also captures the contextual information of each node (i.e., building component) within the overall structure of the graph (i.e., BIM). Fig. 10 depicts the architecture of GraphSAGE implemented in this study. While the figure illustrates a model that considers neighboring nodes up to three hops (i.e., three SAGE convolution layers), the number of SAGE convolution and activation layers, as well as the dimension of each layer, vary across experiments. The optimal configuration for the number of hops and layer dimensions is determined in the second experiment. Each SAGE convolution layer maps the input from the previous layer into a varying dimensional space, followed by a ReLU activation function. When label and one-hot encodings are used, the final output applies a log Softmax function in the linear layer to produce log probabilities.

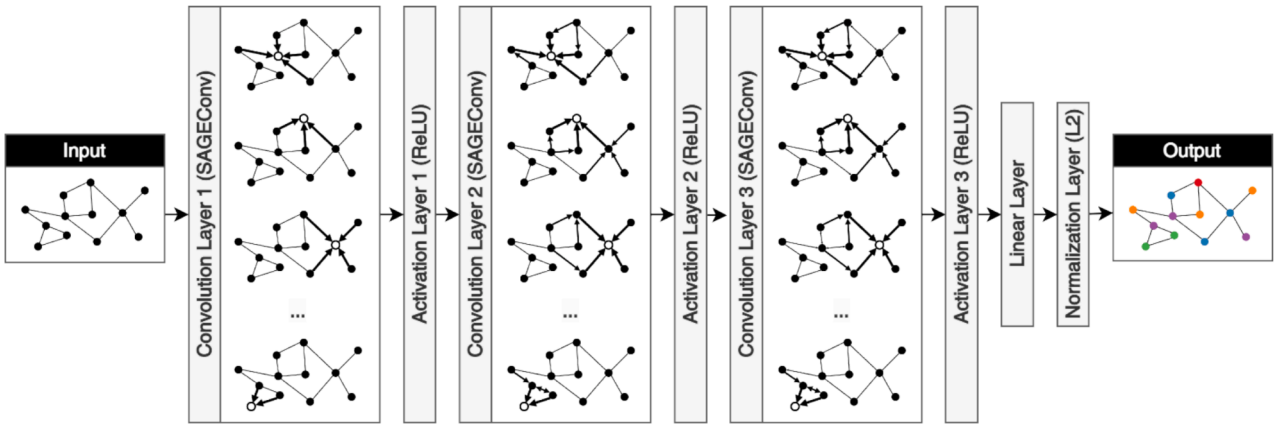


Fig. 10. Node classification process of GraphSAGE with three hops.

4. Experimental design

4.1. Validation dataset

For validation, we collected BIMs from five high-rise residential building projects provided by a major contractor that utilizes the 42 subtypes introduced earlier (Table 5). Each model, referred to as project A, B, C, D, and E, contained 4,691, 4,629, 4,674, 4,573, and 4,348 generic BIM objects (i.e., nodes), respectively. These objects include slabs, walls, beams, columns, stairs, and foundations, which are interconnected by 18,295, 18,840, 16,727, 15,283, and 14,499 edges, respectively. As the subtypes of BIM objects in these models are

Table 5
Distribution of BIM objects by subtype.

Code	Generic Type	Subtype	A	B	C	D	E
0	Wall	Core wall	792	724	577	703	834
1	Wall	Horizontal wall	108	111	110	106	100
2	Wall	Vertical wall	220	218	220	212	202
3	Wall	Perimeter wall	1,158	1,232	1,226	1,350	1,126
4	Wall	Side wall	80	80	78	51	50
5	Wall	Entrance retaining wall	0	0	0	7	20
6	Wall	Loadbearing retaining wall	19	18	22	32	0
7	Wall	Miscellaneous wall	5	7	10	2	5
8	Wall	Unit partition wall	158	122	139	81	80
9	Wall	Non-loadbearing wall	291	302	289	298	221
10	Wall	Auxiliary space wall	0	0	0	0	15
11	Wall	Roof ornamental wall	23	0	23	6	6
12	Wall	Roof parapet wall	14	14	14	14	14
13	Wall	Penthouse parapet wall	19	20	19	18	26
14	Wall	Ramp parapet wall	24	2	16	13	2
15	Wall	Balcony parapet wall	913	899	936	924	909
16	Wall	Miscellaneous parapet wall	33	2	10	5	0
17	Column	Transfer column	0	6	0	0	0
18	Beam	Core beam	212	213	372	205	200
19	Beam	Transfer beam	0	2	0	0	0
20	Beam	Miscellaneous beam	5	4	5	5	3
21	Beam	Wall girder	14	28	6	3	24
22	Beam	Interior lintel	100	104	100	98	97
23	Beam	Non-loadbearing lintel	100	98	99	94	92
24	Stairs	Entrance stairs	4	0	4	2	0
25	Stairs	Interior stairs	57	55	57	53	52
26	Slab	Core slab	51	72	51	2	50
27	Slab	Entrance slab	2	1	2	50	2
28	Slab	Entrance ramp slab	10	0	10	5	0
29	Slab	Piloti slab	4	1	4	2	2
30	Slab	Basement utility pit slab	2	2	2	3	0
31	Slab	Interior slab	53	70	53	50	46
32	Slab	Bathroom slab	200	196	200	147	138
33	Slab	Miscellaneous slab	0	2	0	0	2
34	Slab	Auxiliary space slab	0	1	0	0	4
35	Slab	Penthouse interior slab	4	4	4	4	5
36	Slab	Penthouse core slab	2	2	2	2	2
37	Slab	Roof ornamental slab	1	0	1	6	6
38	Slab	Canopy slab	2	2	2	2	2
39	Foundation	Entrance strip foundation	0	0	0	7	0
40	Foundation	Mat foundation	9	13	9	9	9
41	Foundation	Haunch	2	2	2	2	2
Total number of nodes			**4,691**	**4,629**	**4,674**	**4,573**	**4,348**
Total number of edges			**18,295**	**18,840**	**16,727**	**15,283**	**14,499**

manually labeled and revised by experts throughout the project until completion, they are considered to be accurate. The number of subtypes reveals a skewed distribution, which is typical in practice. Although there were 42 subtypes identified across the five projects, no single project utilizes all 42 subtypes. Instead, the number of subtypes in each project varies, with 35, 35, 35, 37, and 34 subtypes utilized in projects A, B, C, D, and E, respectively. Notably, several subtypes exist only in a single project, such as 'Auxiliary space wall,' 'Transfer column,' and 'Entrance strip foundation.'.

The training and test datasets were constructed through cross-validation across the five projects. In each iteration, one project served as the test data while the remaining four were used as the training data. This cross-validation setup replicates practical scenarios where the proposed method is applied to a new project based on prior projects. The resulting dataset structure is summarized in Table 6, with each column depicting the distribution of training and test data for each subtype included in each iteration of the projects. The distribution reflects realistic scenarios where certain subtypes are more commonly used than others while some subtypes may not have been used in prior projects (i.e., training dataset) but utilized in a new project (i.e., test dataset) as bolded in Table 6.

4.2. Experiments

In this study, four experiments are conducted to verify the applicability of the proposed method in semantic elaboration (Fig. 11):

4.2.1. Approach 1: Node property prediction approach

Experiment I – Predicting requirements for insulation, waterproofing, and loadbearing: The first experiment tests the performance of the proposed method using the node property prediction approach in predicting three functional requirements: insulation, waterproofing, and loadbearing, as shown in Table 4. The hyperparameters utilized in this study were three SAGE convolution layers of dimensions 16, 32, and 64 [48], training a separate model for each functional property. Although this approach could theoretically categorize generic objects into 48 subtypes by combining the three types of functional requirements and six generic types ($2^3 \times 6 = 48$), it could only cover 14 unique distinctions among 42 subtypes listed in Table 4. Therefore, we focused more on the next subtype classification approach.

4.2.2. Approach 2: Node subtype classification approach

The second set of experiments tests the performance of the proposed method using the subtype classification approach considering the following factors:

Experiment II — Comparison of hops and dimensions for subtype classification: The second experiment evaluates the performance of the proposed method utilizing varying model architecture configurations. These configurations include the number of SAGE convolution layers, one to seven hops, and layer dimensions, ranging from 16, 32, 64, 128, 256, 512, 1024, and 2048 (2^3 to 2^{11}). The focus of this experiment is to identify the optimal GraphSAGE model configurations for subtype classification.

Experiment III — Comparison of conventional encoding methods and LLM encoding: The third experiment compares the performance of the conventional encodings (i.e., label and one-hot encodings) and LLM encodings (i.e., embeddings generated using 'text-embedding-3-small' and 'text-embedding-3-large') as data encoding, which are multi-dimensional vectors distributed relatively by their semantics. This experiment aims to identify if providing semantic nuances as encodings for training AI models effectively improves the performance for inferencing the functional requirements of the objects for semantic elaboration.

Experiment IV — Comparison of different models: The fourth experiment evaluates the performance of the proposed relation-based approach, by comparing it against other AI models utilizing feature-

Table 6
Numbers of training data and test data items.

Code	Subtype	Test/Train A	Test/Train B	Test/Train C_	Test/Train D	Test/Train E
0	Core wall	792/2838	724/2906	577/3053	703/2927	834/2796
1	Horizontal wall	108/427	111/424	110/425	106/429	100/435
2	Vertical wall	220/852	218/854	220/852	212/860	202/870
3	Perimeter wall	1158/4934	1232/4860	1226/4866	1350/4742	1126/4966
4	Side wall	80/259	80/259	78/261	51/288	50/289
5	Entrance retaining wall	**0/27**	**0/27**	**0/27**	7/20	20/7
6	Loadbearing retaining wall	19/72	18/73	22/69	32/59	0/91
7	Miscellaneous wall	5/24	7/22	10/19	2/27	5/24
8	Unit partition wall	158/422	122/458	139/441	81/499	80/500
9	Non-loadbearing wall	291/1110	302/1099	289/1112	298/1103	221/1180
10	Auxiliary space wall	**0/15**	**0/15**	**0/15**	**0/15**	**15/0**
11	Roof ornamental wall	23/35	**0/58**	23/35	6/52	6/52
12	Roof parapet	14/56	14/56	14/56	14/56	14/56
13	Penthouse parapet	19/83	20/82	19/83	18/84	26/76
14	Ramp parapet	24/33	2/55	16/41	13/44	2/55
15	Balcony parapet wall	913/3668	899/3682	936/3645	924/3657	909/3672
16	Miscellaneous parapet	33/17	2/48	10/40	5/45	**0/50**
17	Transfer column	**0/6**	**6/0**	**0/6**	**0/6**	**0/6**
18	Core beam	212/990	213/989	372/830	205/997	200/1002
19	Transfer beam	**0/2**	**2/0**	**0/2**	**0/2**	**0/2**
20	Miscellaneous beam	5/17	4/18	5/17	5/17	3/19
21	Wall girder	14/61	28/47	6/69	3/72	24/51
22	Interior lintel	100/399	104/395	100/399	98/401	97/402
23	Non-loadbearing lintel	100/383	98/385	99/384	94/389	92/391
24	Entrance stairs	4/6	**0/10**	4/6	2/8	**0/10**
25	Interior stairs	57/217	55/219	57/217	53/221	52/222
26	Core slab	51/175	72/154	51/175	2/224	50/176
27	Entrance slab	2/55	1/56	2/55	50/7	2/55
28	Entrance ramp slab	10/15	**0/25**	10/15	5/20	**0/25**
29	Piloti slab	4/9	1/12	4/9	2/11	2/11
30	Basement utility pit slab	2/7	2/7	2/7	3/6	**0/9**
31	Interior slab	53/219	70/202	53/219	50/222	46/226
32	Bathroom slab	200/681	196/685	200/681	147/734	138/743
33	Miscellaneous slab	**0/4**	2/2	**0/4**	**0/4**	2/2
34	Auxiliary space slab	**0/5**	1/4	**0/5**	**0/5**	4/1
35	Penthouse interior slab	4/17	4/17	4/17	4/17	5/16
36	Penthouse core slab	2/8	2/8	2/8	2/8	2/8
37	Roof ornamental slab	1/13	**0/14**	1/13	6/8	6/8
38	Canopy slab	2/8	2/8	2/8	2/8	2/8
39	Entrance strip foundation	**0/7**	**0/7**	**0/7**	**7/0**	**0/7**
40	Mat foundation	9/40	13/36	9/40	9/40	9/40
41	Haunch	2/8	2/8	2/8	2/8	2/8

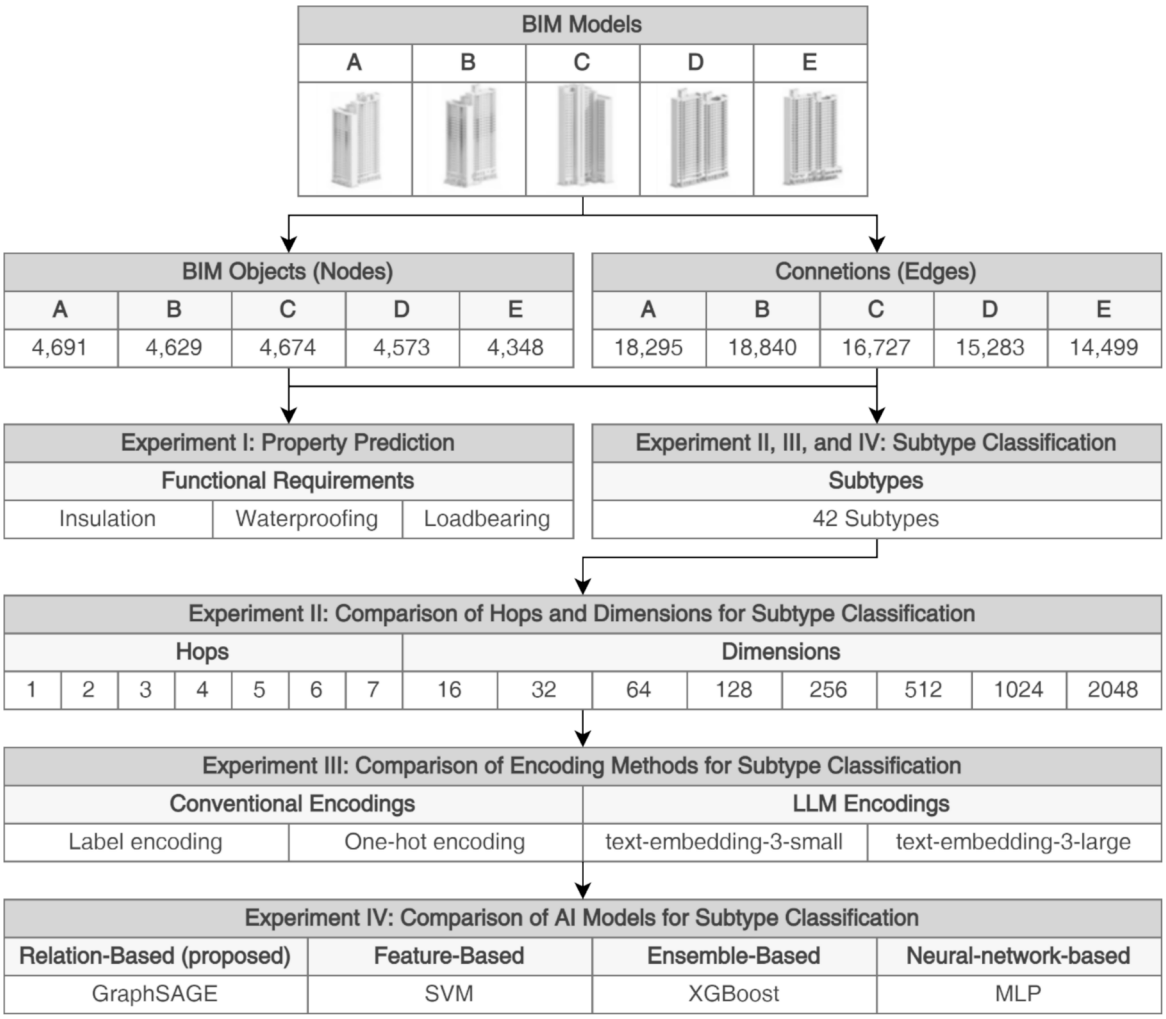


Fig. 11. Experimental design.

based, ensemble-based, and neural-network-based approaches utilized in previous subtype classification studies, namely SVM [9,19,21], XGBoost [20], and MLP [21,23], correspondingly. SVMs are discriminative classifiers designed to maximize the margin between the nearest members of different classes, known as support vectors. They handle both linear and nonlinear separability through the use of kernel tricks, which transform the input data into higher-dimensional spaces where it can be more easily classified [49]. XGBoost is an ensemble technique that boosts weak decision trees to form a robust classifier. It corrects predecessors' errors iteratively and is known for its efficiency and adaptability across various datasets [50]. MLP replicates the neural architecture of the human brain, featuring layers of interconnected nodes or neurons. They are particularly effective in capturing complex patterns and relationships in data [51]. To incorporate BIM objects' connections to train these models, the relationships of objects were presented as tabular data, with columns for each generic type (i.e., walls, columns, beams, slabs, stairs, and foundations) on each hop, which values in the rows indicating the number of connected building components at the hop.

4.3. GraphSAGE training and hyperparameters

For validation, we utilized a workstation equipped with an Intel i7-11700KF processor, 64 GB of random-access memory (RAM), and a 64-bit Windows 10 operating system. The graphical processing unit (GPU) used was an NVIDIA GeForce RTX 3080. The software environment includes Python version 3.11.5 on Revit 2025 for BIM software. For the implementation and training of the GraphSAGE model, we utilized PyTorch and PyTorch Geometric libraries. Table 7 outlines the hyperparameters utilized in each experiment. The learning rate, weight decay, and optimizer were set to 0.005, 0.0001, and AdamW, respectively. The number of epochs was fixed at 4,000, except in the second experiment where we examined accuracy and loss changes across epochs under varying model architecture parameters. While the number of SAGE convolution layers and their dimensions followed the settings from previous work [49] in the first experiment, the configurations for the third and fourth experiments were based on the optimal settings identified in the second experiment. Cross-entropy was used as the loss function for label encoding, while binary cross-entropy with logits and cosine embedding loss were used for one-hot encoding and LLM encoding, respectively. For input and output encoding dimensions, label encodings were set to a dimension of 1. One-hot encodings used dimensions corresponding to the number of generic types and subtypes, 6 and 42, correspondingly. The LLM encodings were adjusted to the optimal layer dimensions identified in the second experiment, compacted from the original dimensions of 1,536 for 'text-embedding-3-small' and '3,072 for text-embedding-3-large,' respectively. SVM and XGBoost for the fourth experiment were implemented using scikit-learn and XGBoost. MLP architecture was implemented using PyTorch, with an input layer, hidden layers with the same number and sizes as GraphSAGE, ReLU activation layer, and an output layer.

Table 7
Hyperparameter configurations for loss function and architecture of GraphSAGE.

Hyperparameters	Experiment I	Experiment II	Experiment III			Experiment IV
			Label encoding	**One-hot encoding**	**LLM encoding**	
Epoch	2,000	4,000	4,000			4,000
Loss function	Cross entropy	Cross entropy	Cross entropy	Binary cross-entropy with logits	Cosine embedding loss	The optimal settings from the third experiment
Number of SAGE convolution layers	3		1 to 7	The optimal number of SAGEConv layers from the second experiment		
Layer dimensions	[16, 32, 64]	2^3 to 2^{11}	The optimal layer dimension from the second experiment			
Encoding methods	Label encoding	Label encoding	Label encoding	One-hot encoding	LLM encoding ('text-embedding-3-small' and 'text-embedding-3-large'	
Input encoding dimensions	1	1	1	6	The optimal layer dimension from the second experiment t	
Output encoding dimensions	1	1	1	42		

4.4. Evaluation Metrics

The F1-score, serving as a balanced measure, combines precision and recall. This metric is particularly effective for imbalanced datasets when weighted by the number of instances in each class (i.e., weighted F1-score), capturing both the model's ability to correctly identify positive instances (precision) and its ability to find all positive instances (recall). The F1-score is calculated as follows:

$$F1-score = \frac{2 \times Precision \times Recall}{(Precision + Recall)}$$

where:

$$Accuracy = \frac{TP + TN}{(TP + TN + FP + FN)}$$

$$Precision = \frac{TP}{(TP + FP)}$$

$$Recall = \frac{TP}{(TP + FN)}$$

here, TP, FP, TN, and FN denote the numbers of true positives, false positives, true negatives, and false negatives, respectively. The weighted F1-score is calculated as:

$$weightedF1-score = \frac{\sum_i F1_i \times N_i}{\sum_i N_i}$$

where N_i is the number of instances for a class i.

This study employs the average weighted F1-score as the primary performance metric to evaluate the results of the first, third, and fourth experiments. To ensure that the performance measure reflects a balanced evaluation across cross-validation folds, the F1-scores from each fold were averaged over the number of projects and weighted as shown in the expression above.

For the second experiment, where we compare the performance of the GraphSAGE models with varying architectural configurations, we used test accuracy by fixing the number of hops and averaging across different layer dimensions to identify the optimal configuration for dimensions. Additionally, we used test accuracy by fixing the dimensions and averaging across different hop counts to identify the best configuration for hop counts.

In addition, statistical tests were conducted to determine the significance of the differences in performance between encoding methods—label, one-hot, and LLM encodings—in the third experiment. We assessed the normality of the differences in F1-scores using the Shapiro-Wilk test [52], with a significance level of $\alpha = 0.05$. When the normality assumption was satisfied ($p > 0.05$), we employed the paired t-test [53] to evaluate the statistical significance of the mean differences between paired observations. In cases where the normality assumption was violated ($p \leq 0.05$), we used the Wilcoxon signed-rank test [54] as a non-parametric alternative. Statistical significance was determined at the $\alpha = 0.05$ level. This means that if the p-value is less than α for the paired t-test and Wilcoxon signed-rank test, we reject the null hypothesis and conclude that there is a statistically significant difference between the encoding methods.

5. Results

5.1. Experiment i – Predicting requirements for insulation, waterproofing, and loadbearing

Table 8 summarizes the performance of the GraphSAGE models trained to predict each functional requirement—insulation, waterproofing, and loadbearing—across cross-validation folds for each project. The F1-scores for insulation range from 0.7859 to 0.9788; waterproofing from 0.8586 to 0.9678; and loadbearing from 0.6735 to 0.9229. All properties recorded their minimum F1-scores when Project A was used as test data and their maximum F1-scores when Project C was used as test data. The average F1-scores are 0.9153 for insulation, 0.9323 for waterproofing, and 0.8504 for loadbearing, resulting in an overall mean F1-score of 0.8933 across all functional requirements. These results indicate that the functional requirements of building objects can be accurately inferred based solely on their connections and generic types. However, they also suggest that performance may vary depending on the specific projects the model is trained on and applied to.

Despite the high performance of the proposed method in predicting functional properties of BIM objects, combining each functional property ($2 \times 2 \times 2 = 8$ possible combinations) with the six generic types theoretically yields 48 unique combinations. However, in practice, only 14 combinations are used, with five for slabs, four for walls, two for beams, and one each for columns, foundations, and stairs. Even when horizontal and vertical contexts are parsed using level properties of the BIM and room classification algorithms, the number of combinations is limited to 33. Nevertheless, 42 subtypes are utilized in practice (as shown in Table 4), incorporating factors that are not mutually exclusive but are distinguished by implicit criteria.

Table 8
Performance of the proposed method in predicting functional requirements.

Functional requirements	A	B	C	D	E	Average F1-score
Insulation	**0.7859**	0.9646	**0.9788**	0.9192	0.9284	0.9153
waterproofing	**0.8586**	0.9534	**0.9678**	0.9475	0.9345	0.9323
loadbearing	**0.6735**	0.8671	**0.9229**	0.9205	0.8681	0.8504

* The highest and lowest F1-scores for each functional requirement are bolded.

Additionally, managing the project using subtypes not only describes the requirements of the properties but also controls the specifications for each property that can be managed collectively. For instance, while both 'Core wall' and 'Entrance retaining wall' require loadbearing without insulation and waterproofing, the 'Core wall' is not determined by horizontal or vertical context, whereas the 'Entrance retaining wall' is located at the entrance and may be situated on the ground floor. Although these subtypes share the same combination of functional properties, the specific loadbearing performance required for each may differ. However, they cannot be managed differently when only binary property requirement prediction is applied. In practice, subtype-based management is preferred for this reason. Therefore, the following experiments are conducted to optimize the subcategorization of BIM objects' subtypes.

5.2. Experiment II – Comparison of hops and dimensions for subtype classification

In the second experiment, we investigated the optimal configuration of the GraphSAGE architecture for subtype classification, focusing on the number of hops (i.e., the relational distance considered in the graph) and the dimension of the SAGE convolution layers. Fig. 12A compares the trends of test accuracy and training loss over epochs for each number of hops, averaged across all layer dimensions and projects. The results reveal that accuracy generally increases with the number of hops up to a certain point. The performance of the GraphSAGE models with one and two hops converge more slowly compared to the models with more hops, requiring around 2,000 epochs with the training loss decreasing, meaning they are still converging. While models with six and seven hops exhibit marginal improvements in accuracy, they significantly increase computational cost multiplied by the size of layer dimension. Considering six or seven hops may encompass almost all nodes in the graph, given the small-world nature of social networks and graphs as suggested by the "six degrees of separation" theory [55], they can lead to overfitting and reduced model generalizability. Therefore, we selected three hops (i.e., three SAGE convolution layers) as the optimal number of hops for the GraphSAGE model, which achieves near-top accuracy at 0.7904 while maintaining reasonable computational costs. Fig. 12B compares the trends of test accuracy and training loss over epochs for each layer dimension, averaged across all numbers of hops and projects. The results indicate that accuracy improves as the layer dimension increases, peaking at a dimension of 1,024 with a test accuracy of 0.7833. Beyond this point, the test accuracy slightly decreases with a dimension of 2,048, which achieves a test accuracy of 0.7797. Based on the trends observed in the training trends, we determined that the optimal configuration for the GraphSAGE model in subtype classification is three hops with 1,024 dimensions per layer.

5.3. Experiment III – Comparison of conventional encoding methods and LLM encoding

Table 9 compares the average F1-score for subcategorization by each subtype using label and one-hot encodings, and LLM encodings with 'text-embedding-3-small' and 'text-embedding-3-large,' commonly utilizing three SAGE convolution layers with 1,024 dimensions as identified optimal form the previous experiment. The results show a general trend of higher performance in predictions of subtypes with a larger number of training and test datasets. Among 42 subtypes, all encoding types scored F1-score at less than 0.1 for six subtypes namely, 'Side wall,' 'Miscellaneous wall,' 'Auxiliary space wall,' 'Transfer column,' 'Miscellaneous slab,' and 'Transfer beam.' Among these subtypes, 'Miscellaneous wall,' 'Auxiliary space wall,' 'Transfer column,' and 'Transfer beam' (four out of six) only exist in one project, and 'Auxiliary space slab' exists in two projects. 'Side wall', which contrasts with 'Perimeter wall,' requiring directional information exhibits a clear drop in performance even with a sufficient number of instances. What is noticeable is that utilizing LLM encoding with 'text-embedding-3-small,' a subtype that does not exist in the training data was accurately predicted with 0.1300 average F1-score for 'Entrance strip foundation.'.

Although performance fluctuates as the number of instances in the training data decreases, there is a clear tendency toward increased performance when semantic-rich LLM encoding with 'text-embedding-3-large' is used compared to conventional encoding methods. The weighted average F1-scores, gradually increase in the order of label encoding, one-hot encoding, and 'text-embedding-3-large,' with values of 0.8026, 0.8475, and 0.8766, respectively. However, the performance drops to 0.7398 when 'text-embedding-3-small' is employed.

Table 10 presents the results of the Shapiro-Wilk normality test indicating the normality assumption was violated in F1-score between encoding types (p-values < 0.05), prompting the use of the non-parametric Wilcoxon signed-rank test for those comparisons. The results reveal that four pairs exempting pairs of 'text-embedding-3-small' against one-hot encoding and 'text-embedding-3-large' are identified to violate the normality assumption, therefore Wilcoxon signed-rank test is employed for those four cases.

The results of the paired *t*-test and Wilcoxon signed-rank test, presented in the p-value matrix in Table 11, reveal that there are no statistically significant differences in model performance when comparing label, one-hot, and the LLM encodings (using 'text-embedding-3-large') although a slight increase in performance in the order of label, one-hot, and LLM encodings' Specifically, there is no significant difference in performance between the label and one-hot encodings ($p = 0.191680$), between label encoding and 'text-embedding-3-large' ($p = 0.234623$), nor between one-hot encoding and 'text-embedding-3-large' ($p = 0.851310$). However, a significant difference is observed between LLM encodings using 'text-embedding-3-small' and 'text-embedding-3-large' ($p = 0.000005$), indicating that improved LLM encoding provided by 'text-embedding-3-large' leads to significant performance gains compared to 'text-embedding-3-small.'.

Overall, the results indicate that while LLM encodings do not bring statistically significant performance gains over conventional encodings, there is a slight increase in performance in the order of label, one-hot, to LLM encodings. Notably, a significant performance increase is observed when comparing 'text-embedding-3-large' to 'text-embedding-3-small,' suggesting that the enhanced semantic representation capability of 'text-embedding-3-large' contributes to improved AI model performance in tasks where semantic nuances are crucial. This finding highlights the potential of advanced semantic encoding in enhancing model inference and suggests that as LLMs continue to evolve, they can further improve the performance of AI models in handling complex, semantically rich tasks.

5.4. Experiment IV – Comparison of different models

The performance of different machine learning models was evaluated to compare the proposed relation-based approach with feature-based, ensemble-based, and neural-network-based approaches. The AI models representing each approach are the proposed method using GraphSAGE (LLM encoding with 'text-embedding-3-large') and those utilized in the previous subtype classification studies, namely, SVM, XGBoost, and MLP. The results are summarized in Table 12. The traditional SVM model achieved a weighted F1-score of 0.4587, demonstrating limited effectiveness in classifying architectural subtypes. In comparison, the ensemble XGBoost model showed a substantial improvement, reaching a weighted F1-score of 0.7543. The MLP model performed similarly, with a weighted F1-score of 0.7369, reflecting a comparable level of effectiveness to XGBoost. Notably, the GraphSAGE model, which incorporates the semantic context of building components using 'text-embedding-3-large,' outperformed all other models, achieving a weighted F1-score of 0.8766.

All AI models scored F1-scores of less than 0.1 for six subtypes: 'Miscellaneous wall,' 'Auxiliary space wall,' 'Entrance strip foundation,'

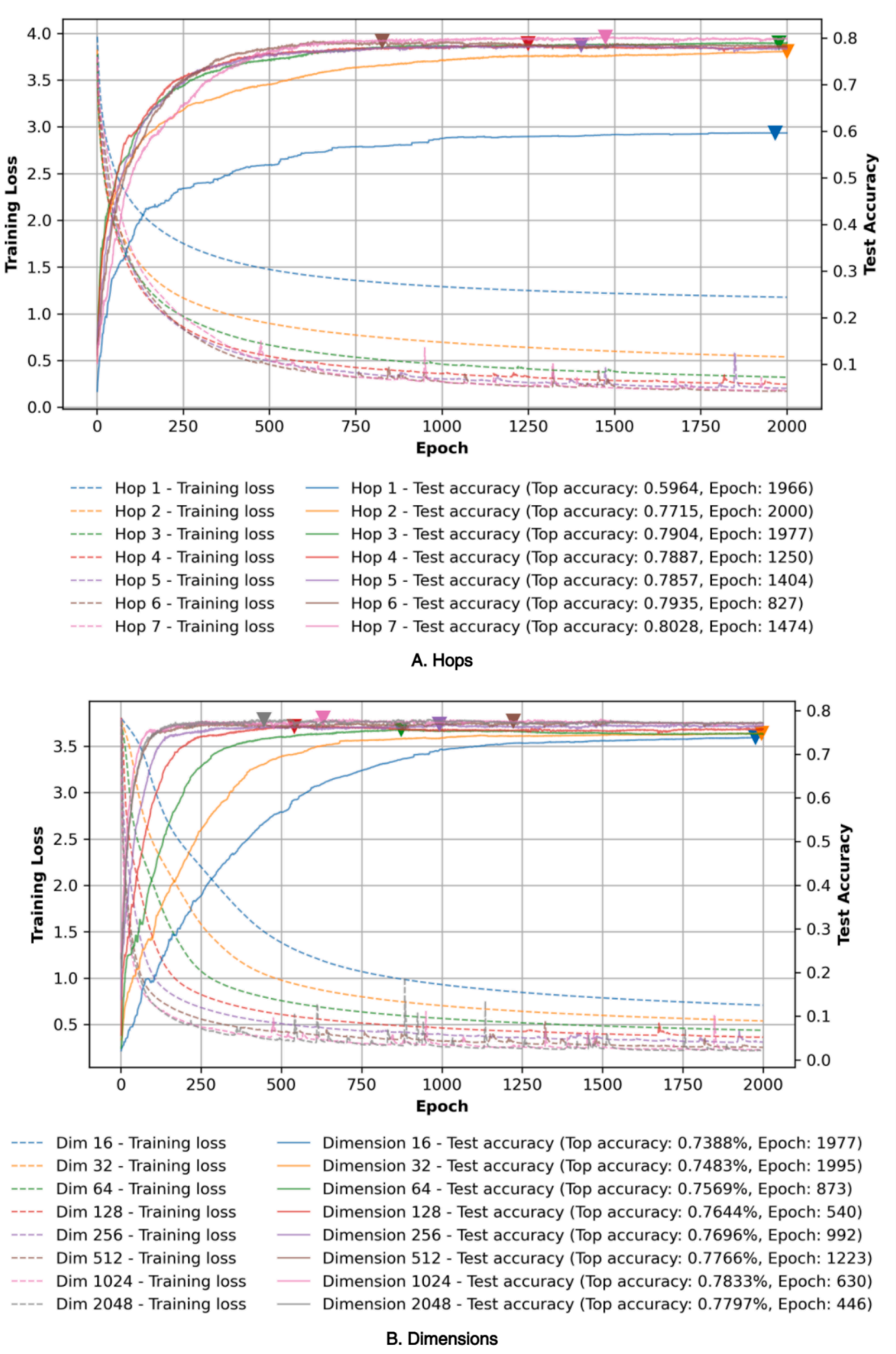


Fig. 12. Comparison of trends of average test accuracy and training loss by epochs for each hop and dimension.

Table 9
Comparison between label, one-hot, and LLM encodings on F1-score for subtype classification.

Code	Subtype	Number of instances	Label encoding	One-hot encoding	LLM encoding	
					text-embedding-3-small	**text-embedding-3-large**
3	Perimeter wall	6092	0.8378	0.8947	0.7517	**0.9236**
15	Balcony parapet wall	4581	0.9274	0.9308	0.7772	**0.9523**
0	Core wall	3630	0.7415	0.8091	0.8387	**0.8695**
9	Non-loadbearing wall	1401	0.7351	0.8298	0.7132	**0.8553**
18	Core beam	1202	0.7741	**0.8872**	0.7112	0.8737
2	Vertical wall	1072	0.7961	0.8076	0.7444	**0.9014**
32	Bathroom slab	881	0.9812	**0.9943**	0.7983	0.9827
8	Unit partition wall	580	0.5187	0.6439	0.5994	**0.6823**
1	Horizontal wall	535	0.8578	**0.9208**	0.7620	0.8735
22	Interior lintel	499	0.9675	0.9556	0.9153	**0.9920**
23	Non-loadbearing lintel	483	0.8544	**0.9073**	0.7373	0.8642
4	Side wall	339	0.0294	**0.0536**	0	0.0073
25	Interior stairs	274	0.9872	0.9583	0.8000	**0.9964**
31	Interior slab	272	0.8332	**0.9808**	0.7646	0.9524
26	Core slab	226	0.6647	0.3805	0.4138	**0.7162**
13	Penthouse parapet	102	0.5620	0.4658	0.2824	**0.5212**
6	Loadbearing retaining wall	91	0.1147	0.3104	0.2673	**0.3489**
21	Wall girder	75	0.6725	**0.8380**	0.6977	0.8184
12	Roof parapet	70	**0.8059**	0.5445	0.7189	0.7668
11	Roof ornamental wall	58	0.4095	**0.4635**	0.2715	0.4572
14	Ramp parapet	57	0.1394	0.5546	**0.6553**	0.5143
27	Entrance slab	57	**0.2821**	0.0382	0.1333	0.0232
16	Miscellaneous parapet	50	0.1306	**0.2267**	0.0667	0.1030
40	Mat foundation	49	0.8932	**0.9440**	0.8916	**0.9440**
7	Miscellaneous wall	29	**0.0667**	0	0	0
5	Entrance retaining wall	27	0	0	**0.1636**	0
28	Entrance ramp slab	25	0.3030	**0.5318**	0.3732	0.4741
20	Miscellaneous beam	22	0.7818	**0.8085**	0.5378	0.7638
35	Penthouse interior slab	21	**0.6416**	0.3943	0.2032	0.3861
10	Auxiliary space wall	15	**0**	**0**	**0**	**0**
37	Roof ornamental slab	14	0.3263	**0.3846**	0.0444	0.2268
29	Piloti slab	13	0.2333	0.5714	**0.6056**	0.5800
24	Entrance stairs	10	0.2914	**0.5714**	0.2000	**0.5714**
36	Penthouse core slab	10	**0.9600**	0.8800	0.2600	0.5933
38	Canopy slab	10	**0.9200**	0.8933	0.6000	0.7333
41	Haunch	10	**1**	0.9143	0.8000	0.9600
30	Basement utility pit slab	9	**0.5476**	0.4667	0.3614	0.4933
39	Entrance strip foundation	7	0	0	**0.1300**	0
19	Transfer column	6	**0**	**0**	**0**	**0**
34	Auxiliary space slab	5	0.0095	**0.0800**	0	0
33	Miscellaneous slab	4	**0**	**0**	**0**	**0**
17	Transfer beam	2	**0**	**0**	**0**	**0**
Weighted average F1-score			0.8026	0.8475	0.7398	**0.8766**

* The highest F1-scores for each subtype and weighted F1-score are bolded.

Table 10
p-values of the Shapiro-Wilk test for identifying the normality of the difference in performance between encoding types.

Shapiro-Wilk test		Label encoding	One-hot encoding	LLM encoding (text-embedding-3-small)
One-hot encoding		0.046930	–	–
LLM encoding	**text-embedding-3-small**	0.022729	**0.057965**	–
	text-embedding-3-large	0.027785	0.000033	**0.433134**

* The p-values larger than the significant level $\alpha = 0.05$, where the normality assumption is violated, are highlighted.

Table 11
p-values of paired *t*-test and Wilcoxon signed-rank test for identifying the significance of the difference in performance between encoding types.

Paired *t*-test/Wilcoxon signed-rank test		Label encoding	One-hot encoding	LLM encoding (text-embedding-3-small)
One-hot encoding		0.143993	–	–
LLM encoding	**text-embedding-3-small**	**0.008129**	**0.000095**	–
	text-embedding-3-large	0.162041	0.700861	**0.000005**

* The p-values larger than the significant level $\alpha = 0.05$ are highlighted.

'Transfer column,' 'Miscellaneous slab,' and 'Transfer beam.' These include five subtypes that exist only in one project and one subtype ('Miscellaneous slab') that exists in two projects, similar to the results of the third experiment.

Compared to the SVM, which failed to outperform any other AI models in any subtype, GraphSAGE excelled particularly in classes with a large number of instances (number of instances equal to or over 100). Among 16 subtypes with a large number of instances, GraphSAGE achieved the highest F1-scores in 10 subtypes. Notably, GraphSAGE outperformed other models in subtypes where topological context is

Table 12
Comparison between SVM, XGBoost, MLP, and GraphSAGE on F1-score for subtype classification.

Code	Subtype	Number of instances	SVM	XGBoost	MLP	GraphSAGE (text-embedding-3-large)
3	Perimeter wall	6,092	0.6511	0.7443	0.7656	**0.9236**
15	Balcony parapet wall	4,581	0.7929	**0.9694**	0.9314	0.9523
0	Core wall	3,630	0.5638	0.6704	0.7345	**0.8695**
9	Non-loadbearing wall	1,401	0	0.6362	0.4372	**0.8553**
18	Core beam	1,202	0.0884	0.6860	0.6505	**0.8737**
2	Vertical wall	1,072	0	0.8546	0.8128	**0.9014**
32	Bathroom slab	881	0.4835	**0.9980**	0.9085	0.9827
8	Unit partition wall	580	0.0906	0.5765	0.5538	**0.6823**
1	Horizontal wall	535	0	**0.9172**	0.8595	0.8735
22	Interior lintel	499	0	0.4711	0.3820	**0.9920**
23	Non-loadbearing lintel	483	0	0.3326	0.3459	**0.8642**
4	Side wall	339	0	**0.0600**	0.0599	0.0073
25	Interior stairs	274	0.0624	0.9944	0.8142	**0.9964**
31	Interior slab	272	0.9520	**0.9943**	0.9907	0.9524
26	Core slab	226	0.0145	0.5862	0.5907	**0.7162**
13	Penthouse parapet	102	0	0.6244	**0.7940**	0.5212
6	Loadbearing retaining wall	91	0	0.3219	0.3133	**0.3489**
21	Wall girder	75	0	**0.9221**	0.7706	0.8184
12	Roof parapet	70	0	0.7938	**0.8431**	0.7668
11	Roof ornamental wall	58	0	0.4227	**0.4726**	0.4572
14	Ramp parapet	57	0	**0.5427**	0.4131	0.5143
27	Entrance slab	57	0	0.3012	**0.3078**	0.0232
16	Miscellaneous parapet	50	0	0.1370	**0.2846**	0.1030
40	Mat foundation	49	0	**0.9636**	0.7598	0.9440
7	Miscellaneous wall	29	0	**0.0571**	0	0
5	Entrance retaining wall	27	0	**0.2000**	0.0364	0
28	Entrance ramp slab	25	0	0.4929	**0.5152**	0.4741
20	Miscellaneous beam	22	0	0.7193	0.3994	**0.7638**
35	Penthouse interior slab	21	0.0800	0.7906	**0.8548**	0.3861
10	Auxiliary space wall	15	**0**	**0**	**0**	**0**
37	Roof ornamental slab	14	0	**0.7714**	0.6364	0.2268
29	Piloti slab	13	0	0.5600	0.4778	**0.5800**
24	Entrance stairs	10	0	0.1600	0.3492	**0.5714**
36	Penthouse core slab	10	0	0.9333	**0.9600**	0.5933
38	Canopy slab	10	0	0.8000	**0.9333**	0.7333
41	Haunch	10	0	**1**	**1**	0.9600
30	Basement utility pit slab	9	0	0.4600	0.4333	**0.4933**
39	Entrance strip foundation	7	**0**	**0**	**0**	**0**
19	Transfer column	6	**0**	**0**	**0**	**0**
34	Auxiliary space slab	5	0	**0.1000**	0.0800	0
33	Miscellaneous slab	4	**0**	**0**	**0**	**0**
17	Transfer beam	2	**0**	**0**	**0**	**0**
Weighted average F1-score			0.4587	0.7543	0.7369	**0.8766**

* The highest F1-scores for each subtype and weighted F1-score are bolded.

clearly defined, such as 'Perimeter wall' (bounding location), 'Interior lintel' (placed between two vertical walls), 'Core wall,' 'Core beam,' 'Core slab,' 'Non-loadbearing wall,' and 'Non-loadbearing lintel.' These subtypes exhibit clear contextual alignment or misalignment with the overall structural frame within a building, enabling GraphSAGE to leverage their spatial and relational characteristics more effectively.

In the 19 subtypes, each containing between 10 and 99 instances, the performance of GraphSAGE was more variable. It achieved the highest F1-scores in 5 out of these 19 subtypes. The MLP performed well in this category, achieving the highest F1-scores in 9 subtypes, while XGBoost led in 7 subtypes. For instance, XGBoost outperformed GraphSAGE in subtypes such as 'Wall girder,' 'Entrance retaining wall,' and 'Roof ornamental slab' by more than 0.1 in F1-score. Similarly, MLP outperformed GraphSAGE in 'Penthouse parapet,' 'Entrance slab,' 'Penthouse interior slab,' 'Penthouse core slab,' and 'Canopy slab,' suggesting that these methods may handle scarce classes more effectively compared to GraphSAGE.

For subtypes with fewer than 10 instances (2 subtypes, excluding subtypes without training data), all models struggled due to the scarcity of data. GraphSAGE achieved the highest F1-score in 'Basement utility pit slab' at 0.4933, while XGBoost achieved 0.1 in 'Auxiliary space slab.' The F1-scores were generally low, indicating the difficulty of classifying subtypes with very few instances.

These results highlight the strength of the proposed relation-based approach, GraphSAGE, in effectively interpreting complex relationships compared to other AI models previously employed for BIM object subtype classification. Simultaneously, the results emphasize that enhancements are required to handle subtypes with fewer instances to improve overall performance in practical scenarios with limited data availability.

6. Discussion

In this section, we further analyze the experimental results, focusing on the differences between projects in which the proposed models underperformed. As identified in the first experiment, the performance of the proposed method varies between projects. Fig. 13 shows the trends of test accuracy and training loss over epochs for each project using the configuration with three hops and 1,024 layer dimensions. It clearly demonstrates that the test accuracy for projects A and B is lower compared to projects C, D, and E, while the training loss exhibits similar overlapping trends among all projects. This indicates that the proposed method struggled to classify BIM objects from projects A and B.

To understand the reasons behind this underperformance, we examined the architectural and topological differences among the projects. Fig. 14 illustrates the BIMs for each project. Projects A, B, and C have a T-shaped building plan, whereas projects D and E have an I-shaped building plan, as depicted in Fig. 14A. Despite the similarity in

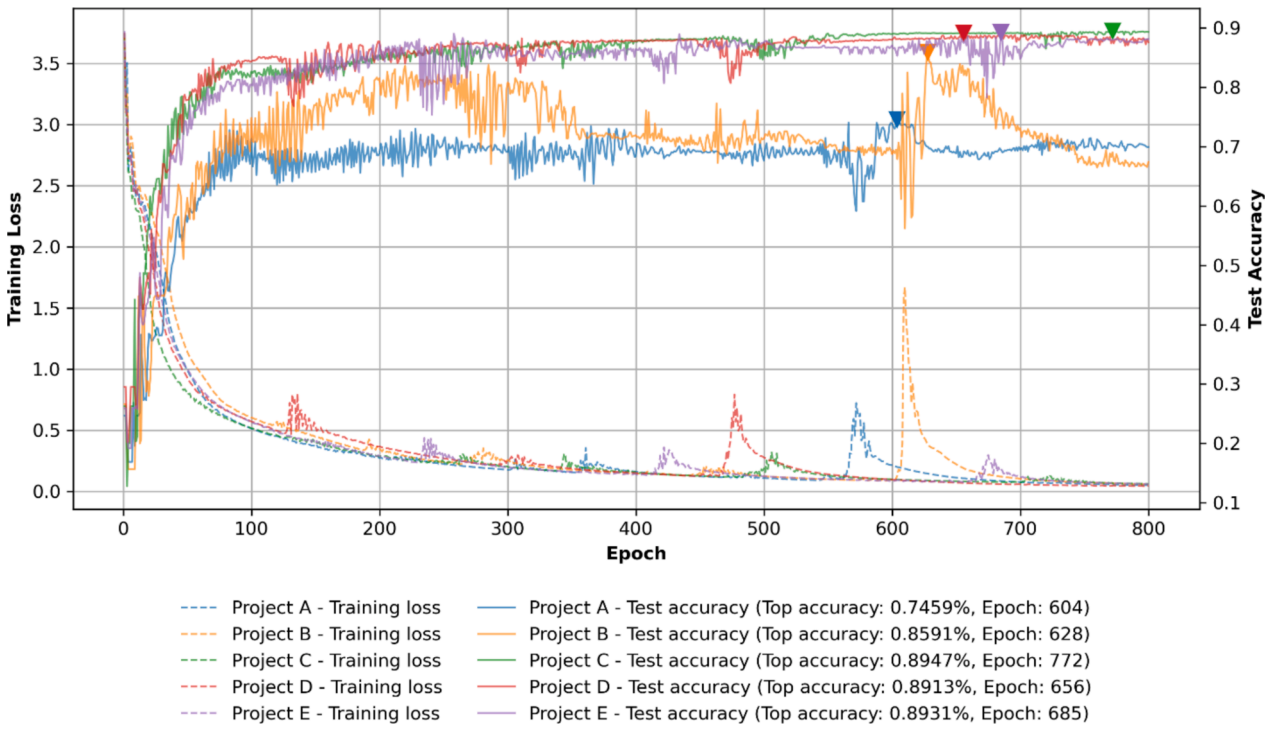


Fig. 13. Comparison of trends of test accuracy and training loss by epochs for each project.

overall topology among projects A, B, and C, there are key differences in the design strategies applied for foundation and roof designs that may have impacted the model's performance (Table 13).

Project A differs from projects B and C in its foundation design, although they are all designed using the 'Mat foundation' subtype. Specifically, project A features a continuous raft foundation covering the entire slab where the building is adjacent to the ground, while projects B and C use pad foundations only partially, as shown in Fig. 14B. In graph terms, the 'Mat foundation' in project A acts as a highly connected node (i.e., a hub), connecting to many other components. This creates a graph with a dense lower section, potentially altering the distribution of node degrees and the overall graph structure. In contrast, the 'Mat foundations' in Projects B and C result in a more distributed foundation network with fewer connections per node. These differences in foundation design affect the horizontal context of the building's topology. Since the foundation is the only generic type that necessarily interacts with the ground, it plays a crucial role in defining the building's base connectivity in the graph. Project B differs from projects A and C in its roof structure. Project B lacks roof structures such as 'Roof ornament walls' and 'Roof ornament slabs', which are present in the other projects, as shown in Fig. 14C. These roof elements provide additional nodes and edges at the upper levels of the graph, contributing to the vertical context of the topology. The absence of these structures in Project B results in a truncated graph at the top, potentially limiting the information available to the model for nodes near the building's upper end.

Considering that the proposed method performance peaked on Project C—which is T-shaped and utilizes pad foundations and roof ornaments that are mostly used—the proposed method performs well on familiar topologies even with few training cases. This emphasizes the need to collect diverse topological datasets reflecting design strategies from previous projects to enhance the model's applicability in practice.

To understand the subtypes for which the proposed method underperformed, we illustrated a scatter plot of the results from the subtype classification model using one-hot encoding, plotting the total number of subtype instances against the F1-score (Fig. 15). The plot is divided into four quadrants based on the mean values of the number of instances per subtype and the mean accuracy.

First quadrant (High Number of Instances / High Accuracy): The top-right quadrant represents subtypes that are both frequently occurring in the dataset and accurately classified by the model. 8 out of 42 subtypes fall into this quadrant. This indicates that the model performs exceptionally well when it has ample data to learn from.

Second quadrant (Low Number of Instances / High Accuracy): The top-left quadrant captures the subtypes that are scarce but still classified accurately. 17 out of 42 subtypes are located here, demonstrating the model's ability to generalize from limited data. The high accuracy in this quadrant suggests that the model can effectively capture and apply the essential features of these subtypes even when examples are sparse.

Third quadrant (Low Number of Instances / Low Accuracy): The bottom-left quadrant includes subtypes that are both infrequent and inaccurately classified. 17 out of 42 subtypes are present in this category, highlighting areas where the model faces challenges. These subtypes likely suffer from insufficient training data or intrinsic difficulties in distinguishing their characteristics.

Fourth quadrant (High Number of Instances / Low Accuracy): The bottom-right quadrant represents frequent subtypes that the model classifies poorly. None of the subtypes are placed in this quadrant.

These observations reveal gaps that need improvement, especially for subtypes with scarce data. We further delved into the subtypes in the third quadrant in Table 14, listing their top three misclassifications while excluding five subtypes that only exist in the test set and not in the training dataset (i.e., 'Miscellaneous wall,' 'Entrance strip foundation,' 'Auxiliary space wall,' 'Transfer column,' 'Miscellaneous slab,' and 'Transfer beam'). The analysis of the classification errors reveals four distinct patterns of confusion, each highlighting specific limitations in the model's current capabilities.

Missing Geometric Dimensions: This type of confusion results from the lack of geometric dimensions included in the graph. For example, the 'Side wall' was mostly classified as 'Perimeter wall,' which dominates in the number of instances included in the dataset. While these subtypes are similar in that they outline the walls of the building, they differ in that the 'Perimeter wall' is longer and the 'Side wall' is shorter. Similarly, 'Penthouse parapet' and 'Ramp parapet' are often classified as

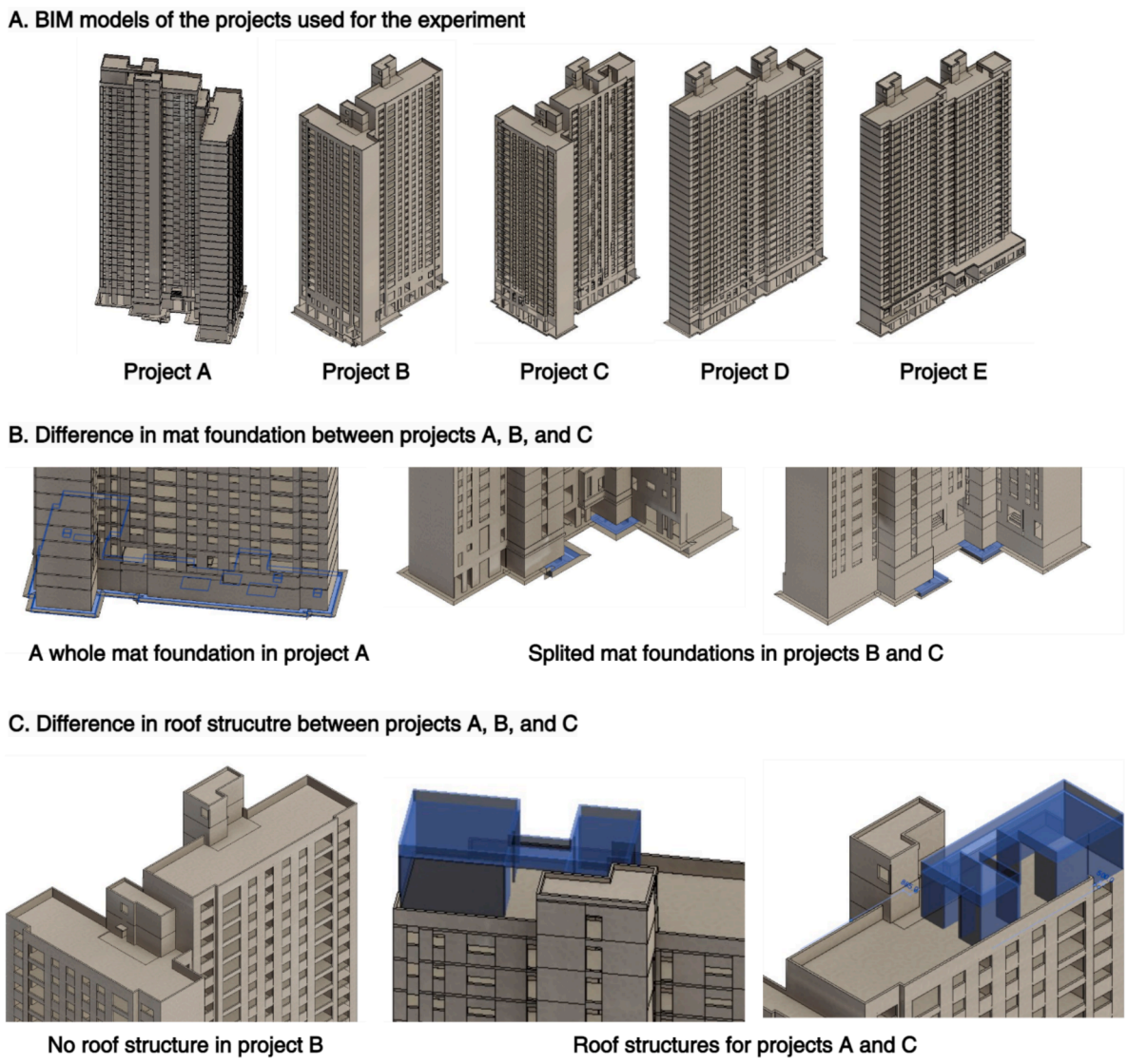


Fig. 14. Visual comparison between BIMs of projects utilized in the experiments.

Table 13
Design strategies utilized in each project.

Projects	Shape	Foundation	Roof ornament
A	T-shape	**Raft foundation**	Exist
B	T-shape	Pad foundation	**Do not exist**
C	T-shape	Pad foundation	Exist
D	I-shape	Pad foundation	Exist
E	I-shape	Pad foundation	Exist

* Unique design strategies for each building project are bolded.

'Perimeter wall.' These misclassifications could be improved by appending geometrical dimensions such as the height of the wall objects. Design parameters, such as the base curve or height of the wall objects, could be included as node attributes to address these limitations in future studies.

Missing Spatial Context: Confusions also occur due to a lack of spatial context, such as room or level, for subtypes that require this information to be distinguished. For example, the 'Entrance slab' was classified as 'Core slab' in 84.21 % of cases. Similarly, the 'Entrance retaining wall' was classified as 'Perimeter wall,' 'Core wall,' and 'Side wall' in 85.19 % of cases. Providing spatial context for entrances may have improved performance, as the model could better comprehend these types as loadbearing and bounding BIM objects. These types of confusions are commonly found, such as when 'Penthouse parapet' and 'Ramp parapet' are classified as 'Balcony parapet,' and 'Penthouse interior slab' is classified as 'Interior slab.' These limitations could be addressed if spatial placement were included as nodes in the graph, which often readily exist in early design stage BIMs.

Incompliance with Generic Types: Despite being provided with generic object types as attributes, the model misclassified 'Roof ornamental wall' as 'Roof ornamental slab' in 17.24 % of cases and 'Roof ornamental slab' as 'Roof ornamental wall' in 71.43 % of cases. This suggests that the proposed method relies more heavily on the topological context of building components rather than the provided attributes. These types of errors could be addressed by incorporating rule-based conditions to constrain the subtypes according to the generic type of the building components.

Taxonomy Limitations for Unclassified Subtypes: While the subtype taxonomy includes various categories that specify functional properties, there will always be building components that do not fit into predefined subtypes. In this study, such components were classified as miscellaneous objects. These subtypes, such as 'Miscellaneous wall,' 'Miscellaneous parapet,' and 'Miscellaneous slab,' often get misclassified as other subtypes, such as 'Miscellaneous parapet' being classified as 'Balcony parapet' or 'Penthouse parapet.' These

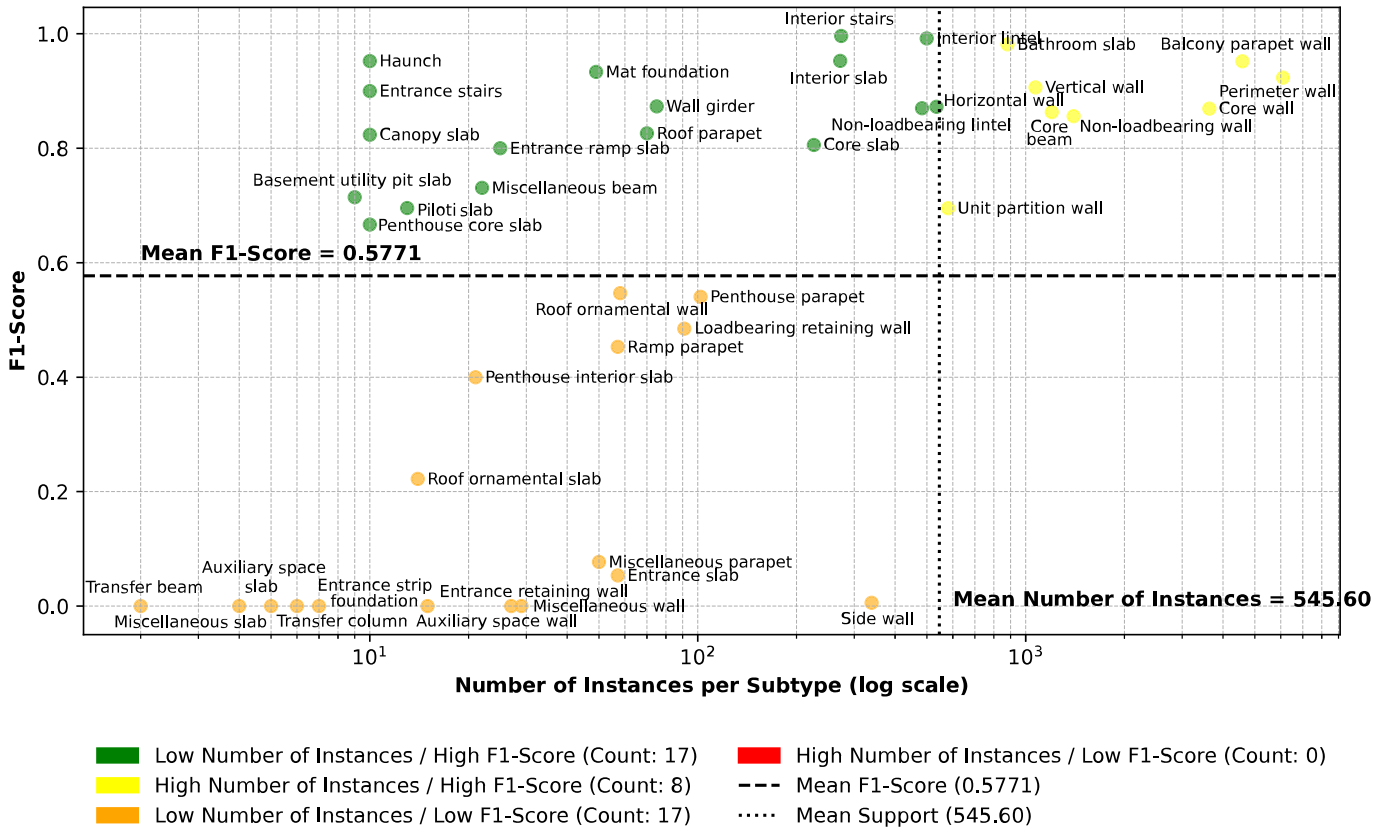


Fig. 15. Scatter plot of the number of training data per code (logarithmic scale) versus F1-score per code.

misclassifications are related to how the taxonomy handles outliers or comprehensively subcategorizes specific cases, rather than being a limitation of the proposed model. This highlights the need for a well-structured and comprehensive subtype taxonomy to effectively manage and accurately classify all building components.

In summary, the analysis reveals that while the proposed GraphSAGE method performs well on subtypes with ample data and a clear distinction of their functional requirements by solely relying on connections between BIM objects, it struggles with subtypes that are scarce in the training dataset. The primary challenges identified include missing geometric dimensions, insufficient spatial context, and reliance on topological cues over attribute-based information. Addressing these issues by incorporating basic geometric information and spatial attributes present in the early design phase, expanding the diversity of training data, and implementing rule-based constraints can enhance the model's ability to accurately classify a broader range of BIM object subtypes. These improvements are essential for deploying robust and reliable AI models in practical applications, where variability in design and data scarcity are common.

7. Conclusion

This study addresses the challenge of automatically inferring the functional requirements of BIM objects in early design stage BIMs with limited LOD, solely utilizing their generic types and connections between them. The proposed method leverages geometric connections between building components using the TETI algorithm introduced in this study, seamlessly transfers BIMs to graphs within a BIM authoring tool, and infers the functional requirements of building components using GNN augmented with a semantically rich encoding method, LLM encoding.

For validation, we utilized BIMs of five high-rise residential buildings from constructed projects, which consist of 6 generic object types with 42 subtypes composed of 3 functional requirements (i.e., insulation, waterproofing, and loadbearing). Four experiments were designed to: 1) assess the proposed method in predicting functional properties, 2) derive the optimal configuration of hops and dimensions required for subtype classification, 3) compare label, one-hot, and LLM encodings, and 4) evaluate the proposed relation-based approach against feature-based, ensemble-based, and neural-network-based approaches.

The first experiment revealed that the proposed method demonstrates strong performance in predicting functional requirements, with average F1-scores of 0.9153 for insulation, 0.9323 for waterproofing, and 0.8504 for loadbearing. Despite this high performance, the results reveal that the combination of functional properties and generic types is limited in reflecting various subtypes, which incorporate non-mutually-exclusive and inclusive properties, including implicit criteria. Additionally, it was found that, in practice, managing BIM objects by grouping subtypes to assign specific performance requirements is preferred.

In the second experiment, we found that a GraphSAGE model configured with three hops and 1,024 layer dimensions achieves optimal performance for BIM object subtype classification. Increasing the number of hops beyond three resulted in negative impacts or only minor accuracy improvements, while significantly raising computational demands, making the chosen configuration the most effective.

The third experiment revealed that LLM encoding, specifically 'text-embedding-3-large,' improves weighted average F1-scores over the label and one-hot encodings, with weighted F1-scores of 0.8026, 0.8475, and 0.8766, respectively, for the label, one-hot encodings, and 'text-embedding-3-large.' Additionally, statistical analysis exhibits a significant improvement when 'text-embedding-3-large' is utilized over 'text-embedding-3-small,' suggesting that semantically-rich representations of classes may increase the performance of AI models when their semantic nuances are crucial.

In the fourth experiment, GraphSAGE, leveraging connections

Table 14
The distribution of the classification errors.

Generic type	Code	Subtype	Results
Wall	4	Side wall	Perimeter wall (0.8083) Balcony parapet (0.1416) Core wall (0.0206)
	5	Entrance retaining wall	Perimeter wall (0.3704) Core wall (0.2593) Horizontal wall (0.2222)
	6	Loadbearing retaining wall	**Loadbearing retaining wall (0.4396)** Core wall (0.2967) Perimeter wall (0.0879)
	11	Roof ornamental wall	**Roof ornamental wall (0.6034)** Roof ornamental slab (0.1724) Perimeter wall (0.0862)
	13	Penthouse parapet	**Penthouse parapet (0.3922)** Balcony parapet (0.3137) Perimeter wall (0.1373)
	14	Ramp parapet	**Ramp parapet (0.4211)** Balcony parapet (0.1930) Perimeter wall (0.1053)
	16	Miscellaneous parapet	**Balcony parapet (0.6200)** Core wall (0.0800) Penthouse parapet (0.0600)
Slab	27	Entrance slab	Core slab (0.8421) **Entrance slab (0.0526)** Interior slab (0.0351)
	34	Auxiliary space slab	Interior slab (0.6000) Entrance slab (0.2000) Piloti slab (0.2000)
	35	Penthouse interior slab	**Penthouse interior slab (0.3333)** Interior slab (0.2857) Core slab (0.2381)
	37	Roof ornamental slab	Roof ornamental wall (0.7143) **Roof ornamental slab (0.2143)** Interior slab (0.0714)

* In the 'Results' column, the bolded text represents correctly classified cases, while non-bolded text represents misclassified cases.

between BIM objects and semantic context, achieved the highest weighted average F1-score of 0.8766, outperforming other AI models previously utilized for subtype classification, including SVM, XGBoost, and MLP, with F1-scores of 0.4587, 0.7543, and 0.7369, respectively. Despite the strength of GraphSAGE, the method reveals existing gaps in handling classes with scarce data, compared to XGBoost and MLP, highlighting room for improvement.

The contributions of this study are as follows. First, this study introduces the concept of semantic elaboration of BIM objects, a topic that has been limitedly addressed and not yet conceptualized in the literature, and proposes specific methods to implement semantic elaboration. The concept of inferring specific information about known BIM objects in early design stage BIMs is expected to enhance the efficiency of the decision-making process. Second, this study demonstrates the applicability of utilizing relationships between building components to infer their functional requirements. Unlike rule-based approaches, the proposed method does not require defining rules for each classification, nor does it rely on complex visual or attribute-based features that may not exist in early design stage BIMs with low LOD. Compared to previous studies that mostly identified ten or fewer subtypes using various features, this study subcategorizes the largest number of subtypes (i.e., forty-two subtypes) managed in practice, based solely on their geometric connections. This approach is expected to improve current practices that require extensive person-hours for reviewing and subcategorizing thousands of BIM objects. Third, we identified the applicability of semantically rich LLM encoding in increasing the performance of AI models for semantic enrichment. This approach is not only limited to domain applications as addressed in this study but can also be generalized to other multi-class classification problems using AI models. Lastly, this study proposes the TETI algorithm to automatically extract geometric connections between BIM objects based on their geometries. This exact yet efficient collision detection method reduces possible errors or omissions in representing the building topology as a computer-interpretable graph, compared to the previous *IfcRelationship*-based approach or bounding-box-based methods commonly utilized in previous studies.

Applying the proposed method in practice can significantly reduce the time-consuming and error-prone task of manually classifying thousands of BIM objects by leveraging semantically elaborated BIMs and screening to detect errors. By automating classification methods, engineers can save time on manual classification and instead focus on defining subtype classifications and effectively managing BIM objects, thereby enhancing overall project management success. This streamlining of the BIM object classification process enables engineers to promptly evaluate the cost and performance of buildings from BIM with low LOD during the early design phase, facilitating timely and informed decision-making.

Despite the demonstrated applicability, the proposed method is not without limitations. First, the method presumes the presence of well-defined subtypes or combinations of properties and the existence of labeled BIMs. Establishing a set of functional requirements that is both mutually exclusive and comprehensively exhaustive is time-consuming and requires extensive domain knowledge, depending on the focus of inference. Second, the experiments were limited to high-rise residential buildings. While topological structures may overlap and repetitively occur within high-rise residential buildings, other building types, such as small housing, may have unique topologies with a smaller number of instances, which may reduce the performance of the proposed method as identified in the discussion. A greater variety of training cases need to be collected and curated. Lastly, the results of the study indicate that there is room for improvement in the input data sources. As discussed in the previous section, geometric information and spatial contexts required for identifying several subtypes were missing, hindering more specific inferences of these subtypes. These limitations are expected to be addressed in future research by enlarging the target project types and securing data diversity, as well as improving model performance in comprehending semantics from BIMs, even with a limited number of training datasets.

The concept of semantic elaboration focuses on automatically enhancing the semantics of early design stage BIMs, where specific features of building components have yet to be determined. The primary challenges identified in this study include missing geometric dimensions, insufficient spatial context, and a reliance on topological cues over attribute-based information. Addressing these issues by incorporating detailed geometric and spatial attributes present in the early design phase, expanding the diversity of training data, and implementing rule-based constraints can enhance the model's ability to accurately classify a broader range of BIM object subtypes. These improvements are essential for deploying robust and reliable AI models that support timely and accurate informed decision-making, where variability in design and data scarcity are common in practical applications.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used OpenAI's tool, ChatGPT in order to proofread the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Suhyung Jang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Conceptualization. **Ghang Lee:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Minkyeong Park:**

Writing – review & editing, Methodology, Investigation, Conceptualization. **Jaekun Lee:** Writing – review & editing, Validation, Methodology, Data curation. **Seungah Suh:** Writing – review & editing, Investigation. **Bonsang Koo:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank Sangyoung Lee, Sanghyo Kim, Seokhyun Choi, Eunchul Ahn, and Hyungyung Ko for their generous provision of time and practical knowledge required for the validation of the proposed framework. This work was supported in 2025 by a Korea Agency for Infrastructure Technology Advancement (KAIA) grant funded by the Ministry of Land, Infrastructure, and Transport (Grant RS-2021-KA163269).

Data availability

Data will be made available on request.

References

[1] P.R. Bredehoeft Jr, C. FAACE, L.R. Dysert, C. Life, T.W. Pickett, C. CEP, J.J. Borowicz, C. PSP, R.B. Brown, P.D.C. Donaldson, Cost Estimate Classification system-as applied in engineering, procurement, and construction for the process industries, (2019). https://aacei-pittsburgh.org/wp-content/uploads/2021/11/cost-estimating-classification-system.pdf (accessed June 21, 2024).
[2] H. Kim, J. Nam, Y. Kim, I. Ryu, Methods for quantitative disassembly and code establishment of CBS in BIM for program and payment management, J. Comput. Struct. Eng. Inst. Korea 36 (2023) 381–389.
[3] J.-K. Song, G.-H. Cho, J.-S. Won, K.-B. Ju, S.-H. Bea, Object classification list for BIM-based maintenance information modeling in electrical and telecommunications field of architecture, J. Korea Acad.-Ind. Coop. Soc. 15 (2014) 3183–3191.
[4] S.-K. Lee, K.-R. Kim, J.-H. Yu, BIM and ontology-based approach for building cost estimation, Autom. Constr. 41 (2014) 96–105, https://doi.org/10.1016/j.autcon.2013.10.020.
[5] F. Boers, S. Lindstromberg, Semantic Elaboration, in: F. Boers, S. Lindstromberg (Eds.), Optim. Lex. Approach Instr. Second Lang. Acquis., Palgrave Macmillan UK, London, 2009: pp. 79–105. Doi: 10.1057/9780230245006_5.
[6] G. Lee, S. Jang, S. Suh, M. Park, H. Roh, L.M. Kim, Three approaches for building information modeling library transplant, in: Integr. Technol. Stemmed Interdiscip. Converg. Res. Lead. Hum.-Friendly Earth, Seoul, Korea, 2023: pp. 234–238.
[7] A. Kiryakov, B. Popov, I. Terziev, D. Manov, D. Ognyanoff, Semantic annotation, indexing, and retrieval, J. Web Semant. 2 (2004) 49–79, https://doi.org/10.1016/j.websem.2004.07.005.
[8] S. Suh, Rule-based subtype classification of apartment BIM elements using geometry and relationship information, Master's Thesis, Yonsei University, 2023.
[9] B. Koo, S. La, N.-W. Cho, Y. Yu, Using support vector machines to classify building elements for checking the semantic integrity of building information models, Autom. Constr. 98 (2019) 183–194.
[10] J. Kim, J. Song, J.-K. Lee, Recognizing and Classifying Unknown Object in BIM Using 2D CNN, in: J.-H. Lee (Ed.), Comput.-Aided Archit. Des. Hello Cult., Springer, Singapore, 2019: pp. 47–57. Doi: 10.1007/978-981-13-8410-3_4.
[11] J. Wu, J. Zhang, New automated BIM object classification method to support BIM interoperability, J. Comput. Civ. Eng. 33 (2019) 04019033, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000858.
[12] B. Koo, R. Jung, Y. Yu, Automatic classification of wall and door BIM element subtypes using 3D geometric deep neural networks, Adv. Eng. Inform. 47 (2021) 101200.
[13] H. Liu, V.J.L. Gan, J.C.P. Cheng, S. (Alexander) Zhou, Automatic fine-grained BIM element classification using Multi-Modal deep learning (MMDL), Adv. Eng. Inform. 61 (2024) 102458, https://doi.org/10.1016/j.aei.2024.102458.
[14] Z. Wang, R. Sacks, T. Yeung, Exploring graph neural networks for semantic enrichment: room type classification, Autom. Constr. 134 (2022) 104039.
[15] Z. Wang, R. Sacks, B. Ouyang, H. Ying, A. Borrmann, A framework for generic semantic enrichment of BIM models, J. Comput. Civ. Eng. 38 (2024) 04023038, https://doi.org/10.1061/JCCEE5.CPENG-5487.
[16] A. Khalili, D.K.H. Chua, IFC-based graph data model for topological queries on building elements, J. Comput. Civ. Eng. 29 (2015) 04014046, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000331.
[17] N. Skandhakumar, F. Salim, J. Reid, R. Drogemuller, E. Dawson, Graph theory based representation of building information models for access control applications, Autom. Constr. 68 (2016) 44–51.
[18] G. Austern, T. Bloch, Y. Abulafia, Incorporating context into BIM-derived data—leveraging graph neural networks for building element classification, Buildings 14 (2024) 527, https://doi.org/10.3390/buildings14020527.
[19] B. Koo, B. Shin, Applying novelty detection to identify model element to IFC class misclassifications on architectural and infrastructure building information models, J. Comput. Des. Eng. 5 (2018) 391–400.
[20] J. Abualdenien, A. Borrmann, Ensemble-learning approach for the classification of Levels Of Geometry (LOG) of building elements, Adv. Eng. Inform. 51 (2022) 101497.
[21] J. Wu, T. Akanbi, J. Zhang, Constructing invariant signatures for AEC objects to support BIM-based analysis automation through object classification, J. Comput. Civ. Eng. 36 (2022) 04022008, https://doi.org/10.1061/(ASCE)CP.1943-5487.0001012.
[22] M. Belsky, R. Sacks, I. Brilakis, Semantic enrichment for building information modeling, Comput.-Aided Civ Infrastruct. Eng. 31 (2016) 261–274, https://doi.org/10.1111/mice.12128.
[23] T. Bloch, R. Sacks, Comparing machine learning and rule-based inferencing for semantic enrichment of BIM models, Autom. Constr. 91 (2018) 256–272, https://doi.org/10.1016/j.autcon.2018.03.018.
[24] L. Ma, R. Sacks, U. Kattel, T. Bloch, 3D object classification using geometric features and pairwise relationships, Comput.-Aided Civ. Infrastruct. Eng. 33 (2018) 152–164, https://doi.org/10.1111/mice.12336.
[25] F. Noichl, F.C. Collins, A. Braun, A. Borrmann, Enhancing point cloud semantic segmentation in the data-scarce domain of industrial plants through synthetic data, Comput.-Aided Civ. Infrastruct. Eng. 39 (2024) 1530–1549, https://doi.org/10.1111/mice.13153.
[26] R. Sacks, L. Ma, R. Yosef, A. Borrmann, S. Daum, U. Kattel, Semantic enrichment for building information modeling: procedure for compiling inference rules and operators for complex geometry, J. Comput. Civ. Eng. 31 (2017) 04017062, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000705.
[27] Y. Yu, K. Lee, B. Koo, K. Lee, Modeling elment relations as structured graphs via neural structured learning to imperove BIM element classification, J. Civ. Environ. Eng. Res. 41 (2021) 277–288, https://doi.org/10.12652/Ksce.2021.41.3.0277.
[28] B. Koo, R. Jung, Y. Yu, I. Kim, A geometric deep learning approach for checking element-to-entity mappings in infrastructure building information models, J. Comput. Des. Eng. 8 (2021) 239–250, https://doi.org/10.1093/jcde/qwaa075.
[29] D. Simeone, S. Cursi, M. Acierno, BIM semantic-enrichment for built heritage representation, Autom. Constr. 97 (2019) 122–137, https://doi.org/10.1016/j.autcon.2018.11.004.
[30] K. Forth, J. Abualdenien, A. Borrmann, Calculation of embodied GHG emissions in early building design stages using BIM and NLP-based semantic model healing, Energy Build. 284 (2023) 112837, https://doi.org/10.1016/j.enbuild.2023.112837.
[31] F.C. Collins, A. Braun, A. Borrmann, Graph-based linking of point cloud and BIM elements for extended DT integration, in: Prof 30th Int Conf. Intell. Comput. Eng. EG-ICE, 2023. https://mediatum.ub.tum.de/doc/1712645/document.pdf (accessed June 22, 2024).
[32] H. Li, J. Zhang, S. Chang, A. Sparkling, BIM-based object mapping using invariant signatures of AEC objects, Autom. Constr. 145 (2023) 104616.
[33] M. Mehranfar, M.A. Vega-Torres, A. Braun, A. Borrmann, Automated data-driven method for creating digital building models from dense point clouds and images through semantic segmentation and parametric model fitting, Adv. Eng. Inform. 62 (2024) 102643, https://doi.org/10.1016/j.aei.2024.102643.
[34] F. Xue, W. Lu, K. Chen, A. Zetkulic, From semantic segmentation to semantic registration: derivative-free optimization–based approach for automatic generation of semantically rich as-built building information models from 3D point clouds, J. Comput. Civ. Eng. 33 (2019) 04019024, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000839.
[35] H. Su, S. Maji, E. Kalogerakis, E. Learned-Miller, Multi-view convolutional neural networks for 3d shape recognition, in: Proc. IEEE Int. Conf. Comput. Vis., 2015: pp. 945–953. https://www.cv-foundation.org/openaccess/content_iccv_2015/html/Su_Multi-View_Convolutional_Neural_ICCV_2015_paper.html (accessed June 27, 2024).
[36] C.R. Qi, H. Su, K. Mo, L.J. Guibas, Pointnet: Deep learning on point sets for 3d classification and segmentation, in: Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2017: pp. 652–660. http://openaccess.thecvf.com/content_cvpr_2017/html/Qi_PointNet_Deep_Learning_CVPR_2017_paper.html (accessed June 27, 2024).
[37] F.C. Collins, M. Ringsquandl, A. Braun, D.M. Hall, A. Borrmann, Shape encoding for semantic healing of design models and knowledge transfer to scan-to-BIM, Proc. Inst. Civ. Eng. - Smart Infrastruct. Constr. 175 (2022) 160–180, https://doi.org/10.1680/jsmic.21.00032.
[38] C.M. Bishop, N.M. Nasrabadi, Pattern recognition and machine learning, Springer, 2006. https://link.springer.com/book/9780387310732 (accessed June 28, 2024).
[39] P. Cerda, G. Varoquaux, B. Kégl, Similarity encoding for learning with dirty categorical variables, Mach. Learn. 107 (2018) 1477–1494, https://doi.org/10.1007/s10994-018-5724-2.
[40] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, in: J. Burstein, C. Doran, T. Solorio (Eds.), Proc. 2019 Conf. North Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. Vol. 1 Long Short Pap., Association for Computational Linguistics, Minneapolis, Minnesota, 2019: pp. 4171–4186. Doi: 10.18653/v1/N19-1423.

[41] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.
[42] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and Efficient Foundation Language Models, (2023). http://arxiv.org/abs/2302.13971 (accessed June 28, 2024).
[43] L. Van der Maaten, G. Hinton, Visualizing data using t-SNE., J. Mach. Learn. Res. 9 (2008). https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf?fbcl (accessed November 19, 2024).
[44] T. Möller, A fast triangle-triangle intersection test, J. Graph. Tools 2 (1997) 25–30, https://doi.org/10.1080/10867651.1997.10487472.
[45] OpenAI, Embeddings, OpenAI Platf. (2024). https://platform.openai.com (accessed July 2, 2024).
[46] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, Matryoshka representation learning, Adv. Neural Inf. Process. Syst. 35 (2022) 30233–30249.
[47] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017). https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html (accessed July 1, 2024).
[48] M. Park, Semantic elaboration of building information modeling objects using a graph neural network, Yonsei University, 2023.
[49] C. Cortes, V. Vapnik, Support-vector networks, Mach. Learn. 20 (1995) 273–297, https://doi.org/10.1007/BF00994018.
[50] T. Chen, C. Guestrin, XGBoost: A Scalable Tree Boosting System, in: Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., ACM, San Francisco California USA, 2016: pp. 785–794. Doi: 10.1145/2939672.2939785.
[51] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (2015) 436–444.
[52] S. Shaphiro, M. Wilk, An analysis of variance test for normality, Biometrika 52 (1965) 591–611.
[53] T. Student, probable error of a mean, Biometrika (1908) 1–25.
[54] F. Wilcoxon, Individual comparisons by ranking methods, in: S. Kotz, N.L. Johnson (Eds.), Breakthr. Stat, Springer, New York, New York, NY, 1992, pp. 196–202, https://doi.org/10.1007/978-1-4612-4380-9_16.
[55] F. Karinthy, Chain-links, in: Struct. Dyn. Netw., Princeton University Press, 2011: pp. 21–26. Doi: 10.1515/9781400841356.21.